

IMPERIAL

Imperial College of Science, Technology and Medicine
Department of Mechanical Engineering

Rocket Landing Control Using Thrust Vectoring

Project Final Report



Author: Oliver McNaughton
Project Supervisor: Dr. Ludovic Renson
Co-Supervisor: Dr. Hamed Rezaee
Number of Pages: 38
Date: June 12, 2025

Abstract

This work presents the design, analysis, and validation of a real-time control system for the vertical landing of a thrust-vectoring reusable rocket. A three-degree-of-freedom (3-DoF) model is derived, incorporating inertial and body-fixed frames, mass/inertia depletion from propellant loss, and time-varying disturbances (wind gusts, propellant slosh, mass uncertainty). A two-loop control architecture combines Lyapunov-based Backstepping with smooth Sliding Mode corrections in both outer (position) and inner (attitude) loops, guaranteeing global asymptotic stability for any positive gain selection. The outer loop computes desired horizontal and vertical thrust components from position errors, mapping them via a simple control allocation into a desired vehicle pitch angle. The inner loop then tracks this angle through engine thrust throttling and gimbal deflection, ensuring rapid attitude convergence without chattering. High-fidelity simulations, driven by smooth trapezoidal-velocity reference trajectories and subjected to ± 30 N/ ± 30 Nm disturbances and 20 % mass error, show sub-meter and sub-degree tracking accuracy within one second of each trajectory phase, disturbance rejection, and actuator commands within defined limits (25–500 N thrust, $\pm 7^\circ$ gimbal). The modular controller is computationally efficient, tuneable, and readily extensible to full 6-DoF scenarios, making it well-suited for embedded flight-computer implementation.

Nomenclature

Symbol	Description	Units
Latin characters		
x, z	Horizontal (x) and vertical (z) position of rocket CoM in inertial frame	m
$\dot{x}, \dot{z}$	Translational velocity components	m s^{-1}
$\ddot{x}, \ddot{z}$	Translational acceleration components	m s^{-2}
θ	Body pitch angle measured from vertical (CCW positive)	rad
$\dot{\theta}, \ddot{\theta}$	Angular rate and acceleration	$\text{rad s}^{-1}; \text{rad s}^{-2}$
T	Engine thrust magnitude	N
ϕ	Gimbal (nozzle) deflection angle (positive right-hand)	rad
F_x, F_z	Thrust components in global x, z axes	N
L	Lever arm: distance from gimbal pivot to CoM	m
m, m_0	Instantaneous mass	kg
J, J_0	Instantaneous inertia about CoM;	kg m^2
g	Gravitational acceleration	m s^{-2}
$\dot{m}, \dot{J}$	Time-rate of change of mass and inertia	$\text{kg s}^{-1}; \text{kg m}^2 \text{s}^{-1}$
α_1, α_2	Mass and inertia coefficients	–
d_x, d_z	Horizontal and vertical disturbance forces	N
m_x	Disturbance moment about CoM	N m
Greek characters		
$\lambda_x, \lambda_z, \lambda_\theta$	Sliding-surface slope gains	–
$\eta_x, \eta_z, \eta_\theta$	Sliding-mode amplitude gains	–
$\psi_x, \psi_z, \psi_\theta$	Boundary-layer half-thickness (SMC smoothing)	–
β	Trajectory blending time	–
Error & control terms		
e_x, e_z, e_θ	Position/attitude tracking errors (state – reference)	–
s_x, s_z, s_θ	Sliding-mode surfaces ($\dot{e} + \lambda e$)	–
$\alpha_x, \alpha_z, \alpha_\theta$	Virtual controls from back-stepping	–
$k_1, \dots, k_8$	Back-stepping proportional/damping gains	–
Actuator limits		
$F_{e,\max}, F_{e,\min}$	Maximum/minimum engine thrust	N
$\phi_{\max}$	Maximum gimbal angle	rad
Subscripts / superscripts		
d	Desired/reference value (e.g. x_d)	–
outer, inner	Quantities associated with outer or inner control loop	–
Abbreviations		
TVC	Thrust Vector Control	–
CoM	Centre of Mass	–
MPC	Model Predictive Control	–
SMC	Sliding Mode Control	–
RLV	Reusable Launch Vehicle	–
VTVL	Vertical Take-off, Vertical Landing	–

Table of Contents

1	Introduction & Literature Review	1
1.1	Current Control Law Landscape for Rocket Landing	2
1.2	Controller Challenges	5
1.3	Objectives	6
1.4	Contributions	7
2	Modelling	8
2.1	Reference Frames	8
2.2	3-DoF Model	8
2.2.1	Equations of Motion	9
2.3	Modelling Disturbances	10
2.4	Propellant Loss Model	11
2.5	State-Space Model	11
2.6	Extension to 6-DoF	12
3	Controller Design	14
3.1	Error Tracking	15
3.2	Inner and Outer Control Loops	15
3.3	Sliding Mode Control Law Derivation	16
3.4	Outer Loop	19
3.4.1	Outer-Loop Backstepping in the vertical z channel	19
3.4.2	Outer-Loop Backstepping for Horizontal Channel (x).	21
3.4.3	Outer Loop Sliding Mode and Backstepping Combined Control Allocation	22
3.5	Inner Loop	23
3.6	Inner-Loop Backstepping in the rotational θ channel	23
3.6.1	Recalculation of F_z	25
3.6.2	Inner Loop Control Allocation	26
3.7	Saturate outputs	26
4	Simulations	28
4.1	Trajectory-Planning for Reference Generation	28
4.2	Simulations	32
4.3	Parameters	34
5	Conclusions & Future work	36
	References	37
	Appendices	39
A	Full Mechanics Derivation	39
A.1	Rotation Matrices	39
A.2	Linear Acceleration	40
A.3	Angular Acceleration	41
A.4	Input matrices	42
B	α_1 & α_2 derivation	43
C	Lyapunov Function and Stability Intuition	44

1 Introduction & Literature Review

Launching material into space comes with substantial cost due to the engineering complexity and resources required for the journey. However, Reusable Launch Vehicles (RLV)¹, and reusable rockets have significantly reduced launch prices by an order of magnitude, transforming orbital access from a state-level privilege into a commercial service[1]. This reuse has made interplanetary exploration, space mining, and deep-space travel considerably more economically viable [2].

Reusing a launch vehicle as well as its boosters is made possible by several methods. The most economically effective has shown to be rocket landing using Thrust Vector Control (TVC)[1]. This method, which SpaceX employed to land their Falcon 9 boosters in 2015 [3] and, more recently, their Starship booster in 2024 [4], actively controls the thrust vector to achieve pinpoint landings. TVC is used to actively control the direction and magnitude of the thrust produced by the boosters' engines. By having full autonomy over the engine thrust, the control system on board can respond to any forces imposed on the rocket, allowing it to vertically land on a launch pad. This technology does have its own set of difficulties, mainly the high computational and engineering resources required to complete such a task. However, even with the listed limitations, TVC has still proven to be the most effective way to reduce launch costs and facilitating a quicker turnaround time between launches compared to traditional booster recovery methods.

This project titled *Rocket Landing Control Using Thrust Vectoring* will be exploring the design and implementation of the control system that would be needed to land a rocket vertically by using TVC. A three-degree-of-freedom (3-DoF) model has been created for the project, which will simulate the rocket's altitude, attitude, and velocity whilst under the effects of aerodynamic drag, gravity, and thrust vectoring forces. A controller has been designed and implemented to land the rocket at a designated site. For simplicity, the model only considers the 'landing burn' portion of flight before touchdown as this is often the most challenging portion to control. Because at this stage there is little aerodynamic damping keeping the vehicle stable and with disturbances, such as wind, producing large attitude deviations that must be corrected solely through thrust-vectoring control inputs, making it a critical test of the proposed control system.

¹In literature and in this text an RLV refers to a space shuttle type vehicle rather than a rocket

1.1 Current Control Law Landscape for Rocket Landing

Classical Control strategies

Classical linear control approaches, such as proportional integral derivative (PID) controllers, are widely recognised and frequently employed by engineers seeking to control systems. Their popularity stems from their user-friendly nature and intuitive tuning mechanisms. Because of this, they have been explored for use in simple vertical landing rocket systems [5]. This paper shows that when tuned correctly, they are able to handle basic stabilisation and hovering of the rocket system. However, if the model increases in complexity, such as introducing propellant slosh, coupled rotational dynamics, disturbances, or even actuator dynamics, PID's become very difficult to tune and effectively. Pure PID control cannot generally guarantee optimal performance or robustness. To achieve better performance, some [6] have used a Linear Quadratic Regulator for control, which provides an optimal state-feedback gain for a given linearised model by minimising a cost function². The LQR approach theoretically yields fast and efficient regulation, by balancing control effort and tracking error.

However, both PID and LQR have their limitations for the specific application of rocket landing control; they assume that there is a linear model (or a linear operating point) and full state feedback. However, when a TVC rocket is landing there are large velocity and attitude changes, which means that in order to achieve global stability, both the PID and LQR have to be tuned at multiple operating points which involves gain scheduling for each scenario [7]. This method can be cumbersome to tune and does not necessarily guarantee robustness against disturbances or varying scenarios which were not accounted for in the tuning process. An LQR in particular can be sensitive to modelling errors and incorporating new dynamics (such as transient actuator behaviour or propellant slosh) into the LQR can prove difficult especially if it requires gain scheduling for multiple scenarios.

Another issue faced by linear controllers is their ability to handle the under-actuated nature of a system of a TVC rocket. Often it requires two loops, an inner loop to control attitude and an outer loop to control position, however once again, tuning becomes difficult. The gain scheduling process becomes increasingly laborious (when a 6-dof model is considered) as this introduces many states and coupled non-linear behaviour. In [5] the system interoperates cold gas thrusters to fully actuate the system, thus doing away with this under-actuated hurdle.

²A cost function defines, in this case, how to balance control action and state error

Therefore, if a linear controller is used for this application it must be in conjunction with a more robust control method to deal with model uncertainty and disturbances.

Optimal Control - Model Predictive Control

Model Predictive Control (MPC) optimises the control inputs over a future time horizon at each step. It has become attractive for implementation in TVC rocket control systems because it can explicitly account for constraints (no fly zones, thrust limits, or actuator angle bounds) whilst performing an optimisation of the control inputs. It is similar to LQR in the way that it will minimise a cost function for a given control input. At each time step it will solve a finite-horizon optimal control problem to find the control inputs which will minimise this cost function³, within constraints. The optimal solution will be applied at that time step, then the optimal control problem will be solved for the next time step and this process repeats until the rocket has landed.

Compared to traditional fixed gain methods of control, MPC can be very adaptive to different initial conditions or even changing trajectories. This can be done by updating internal weights or modifying the cost function in flight. It is able to deal with multi-objective goals (structural/mechanical/tilt constraints can be implemented), which make it an ideal candidate for the rocket landing control problem, especially one where cost is a factor. This type of control has been successfully implemented by [8], the objective was to have a fuel-optimal landing trajectory. By having a fuel optimal landing trajectory it can reduce wear on the engines, feed system, and even leave fuel in the rocket for reuse at a future date, all of which reduce the cost of launch.

The main limitation, as pointed out by [9], is the computational cost required for solving this optimal control problem for each time step. The frequency for solving these optimal control problems must often exceed $50Hz$ (a typical control rate), which can be challenging for compact computational systems. In [9] this problem is remedied by decreasing the time horizon as the rocket approaches the landing site; however, it still incurs a high computational cost.

Non-Linear Control - Backstepping Control

The landing problem of a TVC rocket is an inherently non-linear problem. Given this, non-linear control methods are a natural choice for this. In [10], adaptive back-stepping control, a type of non-linear control, has shown to be effective for use with RLV's. Here the vehicle controls its

³In thrust-vector-controlled rockets, the cost function for MPC simply adds up the rocket's path errors and the amount the nozzles move over the flight, plus a small penalty if it misses the final target, into one number that the guidance system then works to minimize

orientation states using 3 actuators, using a two loop control method with the outer loop controlling the attitude angle subsystem and the inner loop controlling the angular rate subsystem. Backstepping control is especially well suited for this application as the RLV's equations can be arranged into a Strict Feedback Form⁴. When using Backstepping, the controller is designed recursively: intermediate virtual control targets are defined for each subsystem and it 'steps back' through the dynamics, whilst ensuring Lyapunov stability⁵ at each step. This Backstepping procedure effectively 'cancels' out any non-linearities by introducing these virtual control inputs and it means each subsystem behaves like a linear first-order system. The main benefit for this project is the modular nature of Backstepping control, which allows for the introduction of additional system states or dynamics (for example actuator dynamics or sensor delays) without the violation of the Lyapunov Stability proof. In [11] the control is split into two loops to deal with the under-actuated nature of the vehicle, with the outer controlling the attitude angles based on position/orientation errors and the inner loop dealing with angular rate tracking, as was shown to be an effective method for the RLV. However, an issue with Backstepping control, much like LQR control, is that it requires an accurate model to be effective. This is why in [11] a disturbance observer was created to increase robustness against disturbances and uncertainties within the model. The final result here was a stable controller which had improved robustness against unknown parameters.

Robust Control - Sliding Mode Control

Sliding Mode control (SMC) is another non-linear control method which is known for its robust properties, a feature very attractive to this project because of our requirement for disturbance and model uncertainty rejection. To formulate a Sliding Mode controller one must first define a 'sliding surface', which for simplicity we can think of as a line on a 2-D surface with a negative slope passing through the origin, where the x & y axes are typically combinations of state errors. If starting at a random point on the 2D surface, the control law drives the state errors towards the sliding surface then once on the sliding surface it will drive the state errors towards the origin (zero state error). SMC is able to deal with non-linearities in the system by driving the system towards the first order combination of state errors (sliding surface), this means it

⁴A system is said to be in *strict feedback form* if its dynamics can be written in a nested, lower-triangular structure where each subsystem depends only on the states of previous subsystems and the input.

⁵Lyapunov stability is a way to determine whether a system will stay close to a desired state (equilibrium) after being slightly disturbed. If a specially chosen function can be shown to always decrease over time, it indicates that the system will eventually return to that state. This concept will be explored in more detail later in the text.

is also excellent against disturbances and model uncertainties. These are essential properties for a controller controlling a TVC rocket where varying mass, shifts in centre of mass, and external disturbances are present. In [10], a paper which we covered in the non-linear control section utilised adaptive Backstepping. The adaptive portion of the controller is a fast terminal⁶ Sliding Mode controller, which is shown to be effective against external disturbances and model uncertainties. This combination of Backstepping and SMC has shown to be very effective for under-actuated systems with high model uncertainty.

A limitation of Sliding Mode control is the 'chattering' that can occur. This chattering is a high frequency switching of the control input, which can be caused when in close proximity of the sliding surface. This feature of SMC is undesirable for practical implementation of this control method as it can damage actuators or be less accurate due to physical backlash within these actuators. This can be remedied with a smoothing function which decreases the controller gain as the states approach the sliding surface. This is commonly implemented with a $\tanh$ function which starts at a certain distance from the sliding surface. In the case of [12], a time varying boundary layer thickness is implemented with a constant gradient to zero control action at the sliding surface.

1.2 Controller Challenges

Here we outline some of the key challenges and considerations encountered when designing a controller for rocket landing control.

Performance vs Practicality

One of the objectives of this project is to design a control system which can be feasibly implemented. Therefore, there are practical limitations which need to be considered when designing the controller. On one hand we want to have a robust controller that achieves the desired reference trajectory and orientation, whilst maintaining actuator limits. On the other hand, there is a risk of creating a controller which is impractical for implementation in a flight computer. This impracticality can stem from excessive computational requirements or by creating a controller "black box" where the behaviour is difficult to interpret, modify, and ultimately tune. There are resource constraints when running a vehicle testing campaign. As such, there is a require-

⁶The fast terminal in this case means an augmented sliding surface which slows down the motion of the tracking error towards the origin as it approaches, i.e. a decrease in control action as the error approaches zero, which can reduce chances of overshoot or 'chattering' around the origin

ment for the controller to be easily customisable and to have easily understood behaviour. For example, if the actuator dynamics of the TVC gimbal system are not modelled and are seen to be causing unstable behaviour, incorporating these dynamics should not be a difficult task requiring excessive resources to complete.

Under-actuated Dynamics

Landing a TVC rocket poses a challenging underactuation problem: it is like balancing a pencil on one's fingertip, where the pencil is a rocket and your finger is acting as the thrust vectoring system. One of the challenges faced when designing a control system for a vertical landing rocket is the problem that the entire system is under-actuated (fewer control inputs than degrees of freedom in the vehicle), which can lead to complex coupled non-linear dynamics, and non-minimum phase behaviour.⁷ The controller must therefore be able to deal with this type of behaviour and also be robust against any non-linear dynamics originating from external disturbances or a plant model mismatch.

1.3 Objectives

In this work, we develop a two-loop non-linear controller, combining outer-loop Lyapunov-based Backstepping with inner-loop smooth Sliding-Mode corrections, which guarantees global asymptotic stability under mass depletion and large non-linear external disturbances. We evaluate its robustness by exposing the closed-loop system to wind gusts, state-estimation noise, and plant-model mismatches, and demonstrate sub-meter position and sub-degree attitude tracking within one second of each trajectory phase. Finally, we exploit the modular nature of our backstepping and sliding-mode architecture—where new dynamics (such as actuator lags or propellant slosh) can be added without invalidating the Lyapunov stability proof, so that future hardware upgrades require minimal downtime to implement.

⁷Because the engine gimbal is located below the rocket's centre of mass, any commanded sideways translation must start by *tilting* the thrust-line. For a rightward translation the engine first gimbals to the right, producing a force that pushes the base leftward and simultaneously generates a clockwise moment that accelerates the centre of mass *left*. Only after the vehicle has tilted does the main thrust acquire a rightward component and the rocket begins to move in the commanded direction. This obligatory initial motion opposite to the commanded direction is the hallmark of a right-half-plane zero and is therefore called *non-minimum-phase* behaviour. The reader is recommended to visualise this with an upright pen as the rocket and their finger as the TVC.

1.4 Contributions

This project delivers several key advances toward the real-time, robust control of a vertically landing reusable rocket, they are as follows:

- **Underactuated positional control:** A control law is formulated which is able to track the position and attitude of a rocket in 3-DoF despite the underactuated nature, using thrust vector control to stabilise the vertical, horizontal, and attitude motions of the rocket.
- **Guaranteed stability for arbitrary positive gains:** By using a Lyapunov proof at each step we guarantee that the closed loop system is stable for any choice of positive controller gains, which simplifies system tuning.
- **Robustness to disturbances and model uncertainty:** Against extreme disturbances the control system is proven to be stable.
- **Real-vehicle implementation readiness:** The controller has been designed from the ground up for computational efficiency for ease of practical implementation.

2 Modelling

To demonstrate that our controller can guide a rocket safely to touchdown, we first need an accurate representation of its motion. This section lays out the inertial and body-fixed frames, derives the Newton-Euler equations for an underactuated 3-DoF TVC rocket, and highlights how to extend this model to 6-degrees-of-freedom (6-DoF).

2.1 Reference Frames

In rigid-body dynamics, it is essential to distinguish between different *reference frames* and to know how to transform a vector's components from one frame to another. For this project we will be working with:

- A **global (inertial) frame** G , with basis vectors $\{\mathbf{x}_G, \mathbf{z}_G\}$. This frame is fixed (non-accelerating, non-rotating) and serves as the “world” coordinate system.
- A **body (local) frame** B , attached to the moving rocket, with basis vectors $\{\mathbf{x}_B, \mathbf{z}_B\}$. As the body translates and rotates, B moves relative to G .

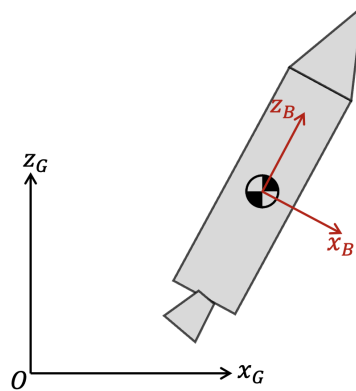


Figure 1: Reference Frames

A reference sketch of these frames is given in Figure 1.

2.2 3-DoF Model

To first understand the underlying dynamics and for rapid prototyping of different control methods, a three-degree-of-freedom (3-DoF) model based on [5] (Figure 2) has been produced. However, with one key distinction, the removal of the cold gas thrusters (represented by forces F_l & F_R) to produce Figure 3. This has been done to mimic the under-actuated nature of a

single engine thrust vector controlled 6-DoF rocket, as there are now three degrees to control $\mathbf{x} = [x \ z \ \theta]^T$ and two control inputs $\mathbf{u} = [T \ \phi]^T$.

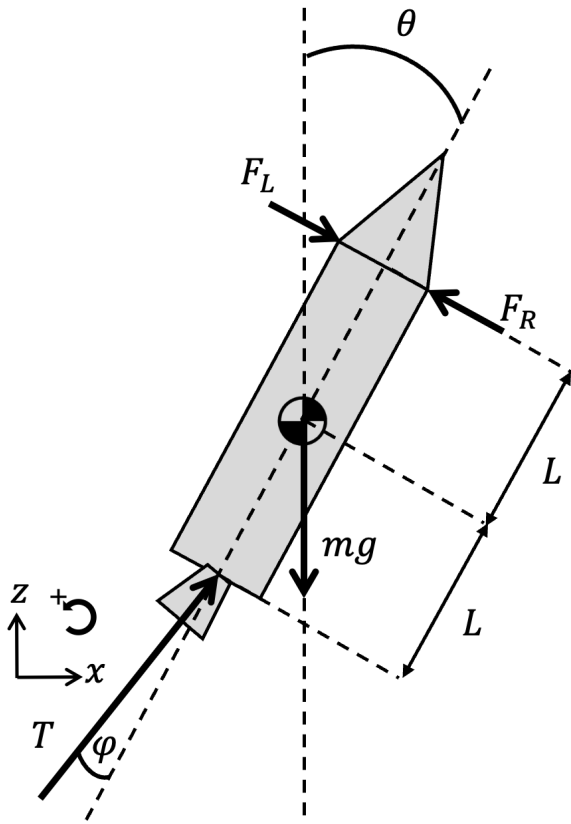


Figure 2: Rocket Model Schematic with Cold Gas Thrusters [5].

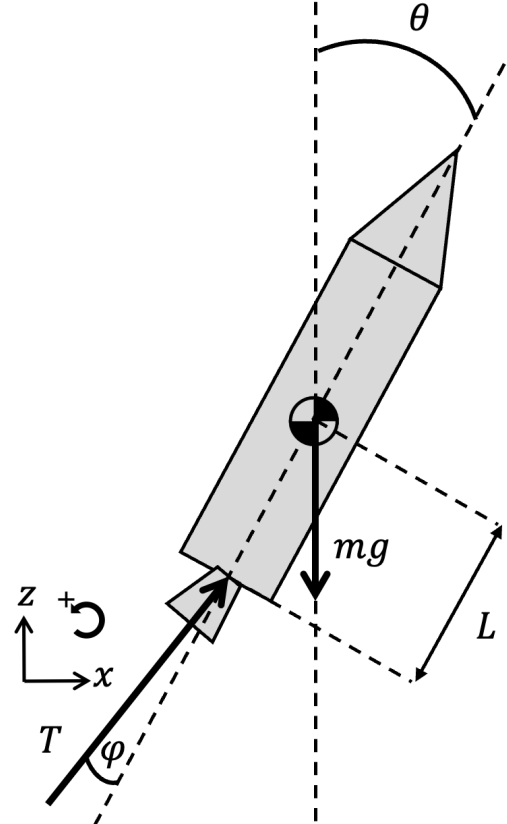


Figure 3: Underactuated 3-DoF Rocket Model Schematic.

2.2.1 Equations of Motion

Using mechanics we obtain the equations of motion for the vehicle. The equation of motion for the angular acceleration relative to the global frame expressed in the global axes is

$$\boxed{{}^G\dot{\boldsymbol{\omega}}_B^G = J_G^{-1} \left({}^G\mathbf{M} - {}^G\boldsymbol{\omega}_B^G \times (J_G {}^G\boldsymbol{\omega}_B^G) \right)}, \quad (1)$$

where ${}^B\boldsymbol{\omega}_B^G$ is the angular velocity of B relative to the inertial frame G , expressed in body-fixed axes. ${}^B\dot{\boldsymbol{\omega}}_B^G$ is the angular acceleration of B relative to G , expressed in B . ${}^B\mathbf{M}$ is the total external moment about the centre of mass, expressed in B . J_B is the inertia tensor of the rocket about its CoM, expressed in B . For linear acceleration in the global frame the equation is as follows:

$$\boxed{{}^G\dot{\mathbf{v}}_{\text{CoM}}^G = \frac{1}{m} {}^G\mathbf{F} = \frac{1}{m} R_{B \rightarrow G} {}^B\mathbf{F}} \quad (2)$$

where ${}^G\dot{\mathbf{v}}_{\text{CoM}}^G$ denotes the CoM acceleration of the body *relative to* G , expressed in the global axes. ${}^G\mathbf{F}$ is the total external force (thrust, drag, etc.) expressed in G . m is the instantaneous mass of the rocket. $R_{B \rightarrow G}(\theta)$ is the rotation matrix which describes the transformation of vectors from the body frame B to the global (inertial) frame G , accounting for the vehicle's orientation θ .

We define the following body-to-global transforms, forces, moments and inertias:

$$R_{B \rightarrow G}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad {}^B\mathbf{F}_{\text{thrust}} = \begin{bmatrix} T \sin \varphi \\ T \cos \varphi \end{bmatrix}, \quad {}^G\mathbf{F}_{\text{grav}} = \begin{bmatrix} 0 \\ -mg \end{bmatrix},$$

$${}^G\mathbf{F} = R_{B \rightarrow G} {}^B\mathbf{F}_{\text{thrust}} + {}^G\mathbf{F}_{\text{grav}} = \begin{bmatrix} T \sin \varphi \cos \theta - T \cos \varphi \sin \theta \\ T \sin \varphi \sin \theta + T \cos \varphi \cos \theta - mg \end{bmatrix},$$

$${}^B M = LT \sin \varphi$$

$$J_B = J, \quad J_G = J$$

$${}^B \dot{\omega} = \dot{\theta}, \quad {}^B \ddot{\omega} = \ddot{\theta}$$

Using Equations 1 and 2 we can obtain expressions of motion for our body (detailed derivation available in Appendix A):

$$\ddot{x} = \frac{T}{m} \sin(\theta + \varphi),$$

$$\ddot{z} = -g + \frac{T}{m} \cos(\theta + \varphi),$$

$$\ddot{\theta} = \frac{L}{J} T \sin \varphi.$$

2.3 Modelling Disturbances

In a real systems disturbances can come in many forms. We will be focusing on the ones specific to the control problem; wind gusts; inaccurate state estimation; and propellant slosh. They can be linear or non-linear, making their nature difficult to control and anticipate. Considering one of the objectives of this project is to provide a robust control system, we must model these disturbances to ensure that our controller remains stable and performs well under realistic operating conditions.

In our model these disturbances will be represented as time-varying non-linear forces and moments, all acting through the centre of mass in their respective axes. We define these

disturbances in the model as follows:

$$\mathbf{d}(t) = \begin{bmatrix} d_x(t) \\ d_z(t) \\ m_x(t) \end{bmatrix} \quad (3)$$

These disturbances are fully defined later in Section 4

2.4 Propellant Loss Model

To mimic the propellant loss experienced by the thrust produced, a model of the change in mass and change in inertia over time has been produced, where α_1 and α_2 are constants. (α_1 and α_2 full derivation in Appendix B).

$$\dot{m} = -\alpha_1 T,$$

$$\dot{J} = -\alpha_2 T.$$

2.5 State-Space Model

For analysis in SIMULINK, the software used for this project, we must put the model into a state-space form. The state space equations are presented in a strict feedback form, as it is required for the Backstepping method. The Backstepping controller will 'backstep' through the states starting with x and finishing with $\dot{\theta}$.

The form is as follows

$$\mathbf{x} = \begin{bmatrix} \dot{\theta} \\ \theta \\ \dot{z} \\ z \\ \dot{x} \\ x \end{bmatrix}, \quad \mathbf{u} = \begin{bmatrix} T \\ \varphi \end{bmatrix}, \quad \mathbf{d}(t) = \begin{bmatrix} d_x(t) \\ d_z(t) \\ m_x(t) \end{bmatrix}.$$

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & \dot{\theta} & -g & \dot{z} & 0 & \dot{x} \end{bmatrix}^T + \begin{bmatrix} 0 & \frac{L}{J} \sin \varphi \\ 0 & 0 \\ \frac{1}{m} \cos(\theta + \varphi) & 0 \\ 0 & 0 \\ \frac{1}{m} \sin(\theta + \varphi) & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} T \\ \varphi \end{bmatrix} + \begin{bmatrix} 0 & 0 & \frac{1}{J} \\ 0 & 0 & 0 \\ 0 & \frac{1}{m} & 0 \\ 0 & 0 & 0 \\ \frac{1}{m} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} d_x(t) \\ d_z(t) \\ m_x(t) \end{bmatrix}. \quad (4)$$

Component form

$$\dot{x} = \dot{x},$$

$$\dot{z} = \dot{z},$$

$$\dot{\theta} = \dot{\theta},$$

$$\ddot{x} = \frac{1}{m} T \sin(\theta + \varphi) + \frac{1}{m} d_x(t),$$

$$\ddot{z} = -g + \frac{1}{m} T \cos(\theta + \varphi) + \frac{1}{m} d_z(t),$$

$$\ddot{\theta} = \frac{L}{J} T \sin \varphi + \frac{1}{J} m_x(t).$$

Note: Traditional strict-feedback form introduces $[x_1, x_2, x_3, x_4, x_5, x_6] = [\dot{\theta}, \theta, \dot{z}, z, \dot{x}, x]$, however, we keep each state written out explicitly to maintain clarity for the reader for the controller section.

2.6 Extension to 6-DoF

To extend this project to a full six degrees of freedom (6-DoF) scenario, two additional actuators are required. The first actuator controls orientation within the newly introduced Z–Y plane, allowing the vehicle to perform pitch and yaw manoeuvres simultaneously. The second actuator governs the roll motion around the vehicle's longitudinal (z-) axis. By allocating roll control to a single actuator, the overall control problem is significantly simplified, since this actuator manages only a single, fully actuated degree of freedom.⁸ Consequently, developing a con-

⁸We do not provide a modelling or control approach for the roll axis in this project, as specific implementations depend heavily on the chosen actuator (cold gas thrusters, reaction wheels etc). Also given that one actuator handles

troller for the 6-DoF vehicle would follow the same fundamental approach demonstrated in this project, extended logically into the additional Z-Y control plane and its respective degrees of freedom.

Chapter Summary

This chapter establishes the mathematical foundation for a TVC-controlled rocket landing by first defining inertial and body-fixed reference frames and then deriving a simplified under-actuated 3-DoF model, using thrust vector and attitude as inputs, via Newton–Euler dynamics. It then incorporates realistic effects (time-varying disturbances and propellant-dependent mass/inertia changes), casts the equations into a strict-feedback state-space form for Backstepping control in Simulink, and outlines how the same approach extends to a full 6-DoF vehicle.

one degree of freedom, the subsystem is fully actuated, therefore control of this axis is typically straightforward.

3 Controller Design

In this chapter we derive the combined Backstepping and Sliding Mode control law. As shown in Figure 4, this controller uses a two loop structure. The outer loop is responsible for tracking the demanded position (x_d, z_d) and the inner loop is responsible for tracking the demanded angular position produced by the outer loop (θ_d). Both Backstepping and Sliding Mode control are Lyapunov based methods (See Appendix C for an intuitive overview of a Lyapunov function), we therefore give an outline of how both control methods utilise their respective Lyapunov functions to guarantee global asymptotic stability.⁹ By the end of the chapter we will have derived a full control law for the hybrid Backstepping-Sliding Mode controller.

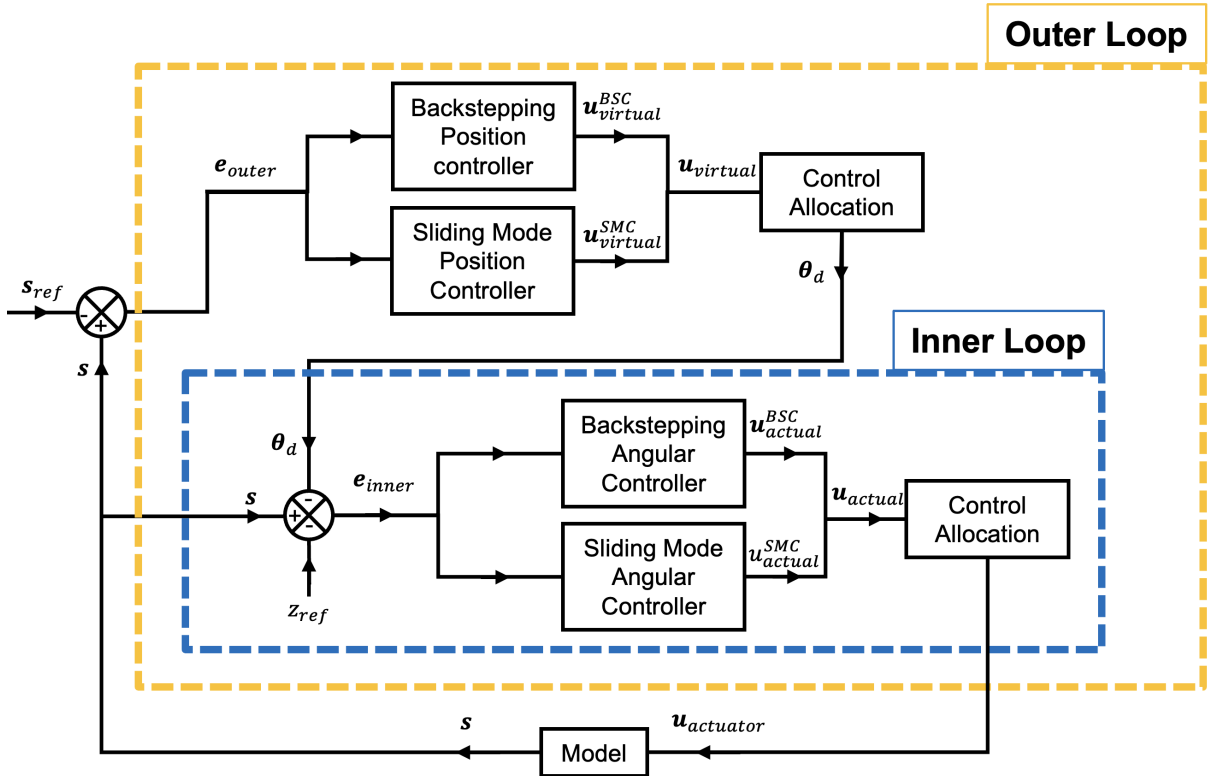


Figure 4: Control Layout.

⁹Global asymptotic stability (GAS) of the closed-loop tracking error in a thrust-vectoring rocket landing system means that, for any admissible initial position, velocity, and attitude error, controller drives the error to zero as time goes to infinity.

3.1 Error Tracking

We define the tracking errors for all controlled states as follows:

$$\mathbf{e} = \mathbf{s} - \mathbf{s}_{\text{ref}} = \begin{bmatrix} e_x \\ \dot{e}_x \\ e_z \\ \dot{e}_z \\ e_\theta \\ \dot{e}_\theta \end{bmatrix} = \begin{bmatrix} x - x_d \\ \dot{x} - \dot{x}_d \\ z - z_d \\ \dot{z} - \dot{z}_d \\ \theta - \theta_d \\ \dot{\theta} - \dot{\theta}_d \end{bmatrix} \quad (5)$$

Here $x, \dot{x}, z, \dot{z}, \theta, \dot{\theta}$ are the measured states and $x_d, \dot{x}_d, z_d, \dot{z}_d, \theta_d, \dot{\theta}_d$ their respective references. These errors form the basis for both the Backstepping and Sliding Mode control laws.

3.2 Inner and Outer Control Loops

The rocket landing problem is inherently under-actuated, meaning the number of actuators is fewer than the degrees of freedom requiring control. Consequently, the translational movements in both horizontal and vertical axes must be managed indirectly through orientation adjustments. This characteristic makes direct simultaneous control of all states challenging. Therefore, the control architecture is logically divided into distinct inner and outer loops.

The outer loop calculates the necessary horizontal and vertical forces based on translational position errors. However, these forces cannot be directly applied due to under-actuation, the system instead achieves the desired force directions by orienting the rocket accordingly to θ_d .

This is then fed into the inner loop, which is tasked with maintaining and rapidly adjusting the rocket's orientation in the global frame (θ) to align the thrust from the engine with the desired directions dictated by the outer loop.

We treat the thrust-vector forces F_x and F_z as “virtual controls” for the translational (outer-loop) subsystem, and subsequently derive the actual actuator commands (thrust magnitude F_e and gimbal angle ϕ) via an inner-loop attitude control. This separation simplifies the overall control problem by clearly distinguishing translational objectives from orientation management, enhancing both robustness and stability of the landing manoeuvre.

One requirement of the inner and outer loop system in real systems, is that the inner loop has a much faster update rate and lower bandwidth, compared to the outer loop, a general rule

of thumb is the ratio of bandwidths should be 10:1 [13]. This is due to the assumption from the Backstepping control law that the inner loop dynamics are "instantaneous". If both loops ran at the same time the outer controller would be trying to correct position errors based on an attitude that has not yet settled.

3.3 Sliding Mode Control Law Derivation

Before going through the derivation of the full control law we derive the smooth Sliding Mode control output used in each loop ($\mathbf{u}_{actual}^{SMC}$, $\mathbf{u}_{virtual}^{SMC}$ in Figure 4).

For each state, the Sliding Mode controller tracks the state error and its derivative, using the control action to drive the states towards a 'sliding surface'. Once on the sliding surface the control action will then 'slide' the state errors along the surface to the origin, a typical path is outlined in Figure 5.

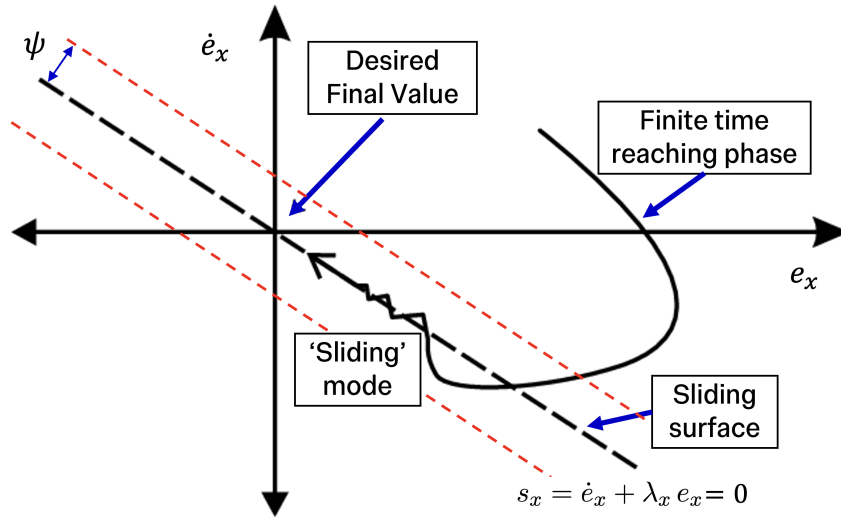


Figure 5: Phase-plane sketch of Sliding Mode control: State trajectories reach the sliding surface $s_x = \dot{e}_x + \lambda_x e_x = 0$ in finite time, then slide along it to the origin. ψ represents the boundary layer thickness, λ represents the slope of the surface.

The problem of 'chattering'¹⁰ which is an issue often experienced by Sliding Mode controllers has been mitigated with a smoothing control law near the surface, this is shown in Equation 8 and explained later in detail. We first define our sliding surface using the tracking error and its derivative to obtain

$$s_x = \dot{e}_x + \lambda_x e_x, \quad \lambda_x > 0.$$

¹⁰This is an undesirable high frequency oscillation of the control action near to the sliding surface caused by discontinuous switching of the control signal; it can increase actuator wear or excite un-modelled dynamics, and therefore must be mitigated

We then choose a Lyapunov candidate for the sliding dynamics,

$$V_x(s_x) = \frac{1}{2} s_x^2.$$

This is positive definite in s_x and vanishes only when $s_x = 0$. We then obtain the differential for V_x as

$$\begin{aligned} \dot{V}_x &= s_x \dot{s}_x \\ &= s_x (\ddot{x} + \lambda_x \dot{x}) \\ &= s_x (\ddot{x} - \ddot{x}_d + \lambda_x (\dot{x} - \dot{x}_d)). \end{aligned} \tag{6}$$

Since our Sliding Mode term u_x^{SMC} enters the vehicle's dynamics as an additional force (or a moment in the case of u_θ^{SMC}), Newton's second law gives

$$m \ddot{x} = u_x^{\text{SMC}} + F_{\text{rest}}(t),$$

where $F_{\text{rest}}(t)$ collects all other modelled and unmodelled forces. Dividing by m yields

$$\ddot{x} = \frac{1}{m} u_x^{\text{SMC}} + \frac{F_{\text{rest}}(t)}{m}.$$

Subtracting the reference acceleration $\ddot{x}_d$ then produces

$$\ddot{x} - \ddot{x}_d = \frac{1}{m} u_x^{\text{SMC}} + \underbrace{\frac{F_{\text{rest}}(t)}{m}}_{w_x(t)} - \ddot{x}_d, \tag{7}$$

so that the SMC input appears directly in $\ddot{x}$ (scaled by $1/m$) and all remaining terms are absorbed into the disturbance $w_x(t)$. By substituting Equation 7 into Equation 6, we obtain

$$\dot{V} = s_x \left[\frac{1}{m} u_x^{\text{SMC}} + w_x(t) \right].$$

where

$$w_x(t) = \underbrace{\frac{F_{x,\text{dist}}(t)}{m}}_{\text{lumped disturbance}} - \ddot{x}_d(t) + \lambda_x \dot{x}(t).$$

Now $w_x(t)$ contains the first Sliding Mode derivative along with the previous disturbances. To

obtain a smooth control law, introduce the continuous “saturation-like” function

$$\phi(s_x) = \tanh\left(\frac{s_x}{\psi_x}\right),$$

and define the Sliding Mode input as

$$u_x^{\text{SMC}} = -m \eta_x \phi(s_x) = m \eta_x \tanh\left(\frac{s_x}{\psi_x}\right). \quad (8)$$

Since

$$\dot{V}_x = s_x \left[\frac{1}{m} u_x^{\text{SMC}} + w_x(t) \right],$$

substituting Equation 8 gives

$$\dot{V}_x = s_x (-\eta_x \tanh\left(\frac{s_x}{\psi_x}\right) + w_x(t)),$$

which is a smooth function of s_x (no discontinuities) and ensures $\dot{V}_x < 0$ whenever $\eta_x > \|w_x\|_\infty$

Exactly the same steps apply for the vertical channel $s_{z,outer} = \dot{e}_z + \lambda_{z,outer} e_z$ and for the attitude channel $s_\theta = \dot{e}_\theta + \lambda_\theta e_\theta$, yielding

$$u_{z,outer}^{\text{SMC}} = -m \eta_{z,outer} \tanh\left(\frac{s_{z,outer}}{\psi_{z,outer}}\right), \quad (9)$$

$$u_{actual}^{\text{SMC}} = u_\theta^{\text{SMC}} = -J \eta_\theta \tanh\left(\frac{s_\theta}{\psi_\theta}\right). \quad (10)$$

These smooth Sliding Mode terms augment the Backstepping commands in each loop and guarantee that each residual s_i converges to zero with no chattering.

We define $\mathbf{u}_{virtual}^{\text{SMC}} = [u_x^{\text{SMC}} \ u_{z,outer}^{\text{SMC}}]^T$ which is a force with units N and u_θ^{SMC} which is a moment with units Nm .

Interpretation of the Sliding Mode constants

The constants λ_x , η_x , ψ_x in the Sliding Mode Control law play the following roles:

- λ_x determines the slope of the sliding surface $s_x = 0$. A larger λ_x places greater weight on position error e_x relative to velocity error $\dot{e}_x$, which accelerates convergence to the surface but may increase overshoot.

- η_x is the Sliding Mode gain, with units of acceleration. It must be chosen larger than the maximum expected disturbance bound $\|w_x\|_\infty$ so that the SMC term always dominates uncertainties and drives $s_x \rightarrow 0$. Increasing η_x improves robustness and reaching speed, at the expense of higher control activity.
- ψ_x sets the boundary-layer thickness around $s_x = 0$. By using $\tanh(s_x/\psi_x)$ instead of $\text{sign}(s_x)$, the control becomes continuous and avoids chattering. A smaller ψ_x yields faster, more aggressive sliding (closer to ideal sign-driven behaviour) but may excite high-frequency dynamics, whereas a larger ψ_x produces smoother but slower convergence.

3.4 Outer Loop

The outer loop takes in $\mathbf{e}_{outer} = [e_x \dot{e}_x e_z \dot{e}_z]^T$ and feeds into both the Backstepping controller and the Sliding Mode controller. In the pure Backstepping (outer-loop) design, we will first derive the Backstepping controller outputs F_x^{BSC} and F_z^{BSC} . These combined will form the signal $\mathbf{u}_{virtual}^{BSC} = [F_x^{BSC} F_z^{BSC}]^T$. This is combined with the signal from the positional Sliding Mode controller to form the total virtual control output from the outer loop as $\mathbf{u}_{virtual} = \mathbf{u}_{virtual}^{BSC} + \mathbf{u}_{virtual}^{SMC}$.

$\mathbf{u}_{virtual}$ will be passed through the outer loop control allocation equations to give the desired body angle of the vehicle θ_d for feeding into the inner loop.

3.4.1 Outer-Loop Backstepping in the vertical z channel

Outlined below is the process for deriving the Backstepping control law for the outer loop positional controller in the z channel. Referring to Equation 4 where we have a strict feedback form. We apply a two-step Backstepping design to stabilize the vertical position z then 'backstep' to stabilise $\dot{z}$. The overall goal is to drive e_z and $\dot{e}_z$ to zero. We first choose a Lyapunov "seed" for e_z

$$V_z = \frac{1}{2} e_z^2.$$

This is positive definite in e_z and will serve as the first stage of Backstepping. We then differentiate V_z and substitute in $\dot{e}_z$

$$\dot{V}_z = e_z \dot{e}_z = e_z (\dot{z} - \dot{z}_d).$$

To ensure $\dot{V}_z$ can be made negative definite in e_z , define a virtual control α_z

$$\alpha_z = \dot{z}_d - k_1 e_z, \quad k_1 > 0.$$

Substituting this into $\dot{V}_z$ gives

$$\dot{V}_z = e_z [-k_1 e_z + (\dot{z} - \alpha_z)] = -k_1 e_z^2 + e_z s_z, \quad (11)$$

where we have defined the *residual error*

$$s_z = \dot{z} - \alpha_z.$$

By choosing $\alpha_z = \dot{z}_d - k_1 e_z$, the term $-k_1 e_z^2$ is negative definite in e_z . However, the cross term $e_z s_z$ remains uncontrolled. To control the cross term we extend V_z to capture both e_z and s_z , this gives a new Lyapunov function

$$V_z^{\text{new}} = \underbrace{\frac{1}{2} e_z^2}_{V_z} + \underbrace{\frac{1}{2} m s_z^2}_{\text{new term}},$$

where m is added to the new term since s_z represents a velocity and the new term represents kinetic energy.¹¹ This combined candidate will allow us to backstep further and solve for the virtual thrust command F_z^{virtual} . We once again differentiate the Lyapunov function to obtain

$$\dot{V}_z^{\text{new}} = e_z \dot{e}_z + m s_z \dot{s}_z.$$

Substituting in

$$\dot{e}_z = s_z - k_1 e_z, \quad \dot{s}_z = \ddot{z} - \dot{\alpha}_z,$$

gives the expression for

$$\dot{V}_z^{\text{new}} = e_z (s_z - k_1 e_z) + m s_z (\ddot{z} - \dot{\alpha}_z).$$

¹¹Refer to Section C for further explanation

Since our Backstepping control action is a virtual force in the vertical channel $F_{z,outer}^{BSC}$ we can use Newtons second law in z to obtain an expression for $\ddot{z}$

$$\ddot{z} = \frac{1}{m} F_{z,outer}^{BSC} - g$$

we then substitute $\ddot{z}$ into $\dot{V}_z^{new}$ to obtain

$$\dot{V}_z^{new} = e_z(s_z - k_1 e_z) + m s_z \left(-g - \dot{\alpha}_z + \frac{1}{m} F_{z,outer}^{BSC} \right) = -k_1 e_z^2 + s_z(e_z - m(g + \dot{\alpha}_z) + F_{z,outer}^{BSC}). \quad (12)$$

By specifying the target for $\dot{V}_z^{new}$

$$\dot{V}_z^{new} = -k_1 e_z^2 - k_2 m s_z^2, \quad k_2 > 0 \quad (13)$$

we can compare Equation 12 and Equation 13, to obtain an expression for $F_{z,outer}^{BSC}$ for $s_z \neq 0$

$$F_{z,outer}^{BSC} = m(g + \dot{\alpha}_z) - k_2 m s_z - e_z.$$

Thus we obtain exactly our target

$$\dot{V}_z^{new} = -k_1 e_z^2 - k_2 m s_z^2, \quad k_1, k_2 > 0.$$

Since both $-k_1 e_z^2$ and $-k_2 m s_z^2$ are negative definite, overall stability of (e_z, s_z) is guaranteed.

The outer-loop Backstepping control output for the vertical channel is therefore

$$\begin{aligned} F_{z,outer}^{BSC} &= m(g + \dot{\alpha}_z) - k_2 m s_z - e_z \\ &= m(g + \ddot{z}_d - k_1 \dot{e}_z) - k_2 m (\dot{e}_z + k_1 e_z) - e_z \\ &= m(g + \ddot{z}_d) - m(k_1 + k_2) \dot{e}_z - (m k_1 k_2 + 1) e_z. \end{aligned}$$

3.4.2 Outer-Loop Backstepping for Horizontal Channel (x).

The process for deriving the outer loop horizontal x channel is exactly the same as the vertical z channel. Therefore we will just outline the mapping needed for the process outlined in the vertical channel control law above to obtain F_x^{BSC}

$$e_z \rightarrow e_x = x - x_d,$$

$$\dot{e}_z \rightarrow \dot{e}_x = \dot{x} - \dot{x}_d,$$

$$\alpha_z \rightarrow \alpha_x = \dot{x}_d - k_3 e_x,$$

$$s_z \rightarrow s_x = \dot{x} - \alpha_x,$$

$$\dot{s}_z \rightarrow \dot{s}_x = \ddot{x} - \dot{\alpha}_x,$$

$$\ddot{z} \rightarrow \ddot{x} = \frac{1}{m} F_x^{BSC}.$$

Target expression:

$$\dot{V}_x^{\text{new}} = -k_3 e_x^2 - k_4 m s_x^2, \quad k_3, k_4 > 0.$$

Control Law:

$$\begin{aligned} F_x^{\text{BSC}} &= m \dot{\alpha}_x - k_4 m s_x - e_x \\ &= m(\ddot{x}_d - k_3 \dot{e}_x) - k_4 m(\dot{e}_x + k_1 e_x) - e_x \\ &= m \ddot{x}_d - m(k_3 + k_4) \dot{e}_x - (m k_3 k_4 + 1) e_x. \end{aligned}$$

3.4.3 Outer Loop Sliding Mode and Backstepping Combined Control Allocation

From our total signal output for the outer loop we have

$$\mathbf{u}_{\text{virtual}} = \mathbf{u}_{\text{virtual}}^{\text{BSC}} + \mathbf{u}_{\text{virtual}}^{\text{SMC}} = \begin{bmatrix} F_z^{\text{total}} \\ F_x^{\text{total}} \end{bmatrix} = \begin{bmatrix} F_z^{\text{BSC}} \\ F_x^{\text{BSC}} \end{bmatrix} + \begin{bmatrix} F_z^{\text{SMC}} \\ F_x^{\text{SMC}} \end{bmatrix}$$

Let F_x^{total} be the desired horizontal force and F_z^{total} the desired vertical force from the outer loop.

We define

$$\theta_d = \arctan 2(F_{x,\text{total}}, F_{z,\text{total}}).$$

Equivalently,

$$\sin \theta_d = \frac{F_{x,\text{ref}}}{\sqrt{F_{x,\text{ref}}^2 + F_{z,\text{ref}}^2}}, \quad \cos \theta_d = \frac{F_{z,\text{ref}}}{\sqrt{F_{x,\text{ref}}^2 + F_{z,\text{ref}}^2}}.$$

Thus, if the engine produces total thrust magnitude

$$T = \sqrt{F_{x,\text{ref}}^2 + F_{z,\text{ref}}^2},$$

and the body rotated by $\theta = \theta_d$ in the global axes (whilst maintaining zero angle of φ), then the components in global axes satisfy

$$F_x = T \sin \theta_d = F_x^{total}, \quad F_z = T \cos \theta_d = F_z^{total}.$$

which satisfies the outer loop.

3.5 Inner Loop

Our inner loop takes in $\mathbf{e}_{inner} = [e_\theta \ \dot{e}_\theta \ e_z \ \dot{e}_z]^T$ and it feeds in into the Backstepping and Sliding Mode controller for attitude control. In the Backstepping inner-loop design, we will first derive the Backstepping control outputs $u_{actual}^{BSC} = M^{BSC}$, which will be combined with the Sliding Mode control output for the θ channel from Equation 10 to give the overall control output $u_{actual} = u_{actual}^{BSC} + u_{actual}^{SMC}$. The z channel errors $e_z, \dot{e}_z$ are required for the inner loop due to the inner loop running much faster than the outer loop (refer to Section 3.2 for explanation). This results in the inner loop shaping both F_x (from the attitude controller control moment) and F_z simultaneously to reach the desired attitude θ .

3.6 Inner-Loop Backstepping in the rotational θ channel

Outlined below is the process for deriving the Backstepping control law for the outer-loop attitude controller in the θ channel. It has similar process to the Backstepping procedure used in the outer loop.

Referring to Equation 4 where we have a strict-feedback form, we apply a two-step Backstepping design to stabilize the attitude angle θ and then “backstep” to stabilize $\dot{\theta}$. The overall goal is to drive e_θ and $e_{\dot{\theta}}$ to zero.

We first choose a Lyapunov “seed” for e_θ

$$V_\theta = \frac{1}{2} e_\theta^2.$$

This is positive definite in e_θ and will serve as the first stage of Backstepping. We then differentiate V_θ and substitute in $\dot{e}_\theta$

$$\dot{V}_\theta = e_\theta \dot{e}_\theta = e_\theta (\dot{\theta} - \dot{\theta}_d).$$

To ensure $\dot{V}_\theta$ can be made negative definite in e_θ , define a virtual control α_θ

$$\alpha_\theta = \dot{\theta}_d - k_5 e_\theta, \quad k_5 > 0.$$

Substituting this into $\dot{V}_\theta$ gives

$$\dot{V}_\theta = e_\theta [-k_5 e_\theta + (\dot{\theta} - \alpha_\theta)] = -k_5 e_\theta^2 + e_\theta s_\theta, \quad (14)$$

where we have defined the *residual error*

$$s_\theta = \dot{\theta} - \alpha_\theta.$$

By choosing $\alpha_\theta = \dot{\theta}_d - k_5 e_\theta$, the term $-k_5 e_\theta^2$ is negative definite in e_θ . However, the cross term $e_\theta s_\theta$ remains uncontrolled. To control the cross term we extend V_θ to capture both e_θ and s_θ , yielding a new Lyapunov function

$$V_\theta^{\text{new}} = \underbrace{\frac{1}{2} e_\theta^2}_{V_\theta} + \underbrace{\frac{1}{2} J s_\theta^2}_{\text{new term}},$$

where J appears because s_θ has units of angular velocity and the new term represents rotational kinetic energy.¹² We then differentiate:

$$\dot{V}_\theta^{\text{new}} = e_\theta \dot{e}_\theta + J s_\theta \dot{s}_\theta.$$

Substituting

$$\dot{e}_\theta = s_\theta - k_1 e_\theta, \quad \dot{s}_\theta = \ddot{\theta} - \dot{\alpha}_\theta,$$

gives

$$\dot{V}_\theta^{\text{new}} = e_\theta (s_\theta - k_1 e_\theta) + J s_\theta (\ddot{\theta} - \dot{\alpha}_\theta).$$

Since our Backstepping control action enters through a moment command M as result of the F_x component of the thrust force, we use the rotational dynamics to obtain an expression for $\ddot{\theta}$

$$\ddot{\theta} = \frac{L}{J} M,$$

¹²Refer to Appendix C for further explanation.

which we substitute into $\dot{V}_\theta^{\text{new}}$, yielding

$$\dot{V}_\theta^{\text{new}} = e_\theta(s_\theta - k_1 e_\theta) + J s_\theta \left(\frac{L}{J} M - \dot{\alpha}_\theta \right) = -k_1 e_\theta^2 + s_\theta(e_\theta + LM - J \dot{\alpha}_\theta). \quad (15)$$

By specifying the target

$$\dot{V}_\theta^{\text{new}} = -k_5 e_\theta^2 - k_6 J s_\theta^2, \quad k_6 > 0,$$

and comparing with (15), one solves for the Backstepping moment command M such that

$$e_\theta + LM - J \dot{\alpha}_\theta = -k_6 J s_\theta.$$

This yields the outer-loop Backstepping control law for the attitude channel:

$$M^{BSC} = \frac{J \dot{\alpha}_\theta - e_\theta - k_6 J s_\theta}{L}.$$

3.6.1 Recalculation of F_z

Because our inner loop runs at a much faster rate than our outer loop, where the outer loop is effectively quasi-static compared to the inner loop. We need to therefore recalculate the F_z command which will be send to the control allocator¹³. This is to ensure that both the F_z and F_x commands are shaped simultaneously to control the rockets attitude θ .

The control law for the inner loop z channel is exactly the same as the outer loop control law, however, there are different control gains needed as the inner loop runs at a much faster time scale.

$$\begin{aligned} F_{z,inner}^{BSC} &= m(g + \dot{\alpha}_z) - k_8 m s_z - e_z \\ &= m(g + \ddot{z}_d - k_7 \dot{e}_z) - k_8 m (\dot{e}_z + k_7 e_z) - e_z \\ &= m(g + \ddot{z}_d) - m(k_7 + k_8) \dot{e}_z - (m k_7 k_8 + 1) e_z, \quad k_7, k_8 > 0 \end{aligned}$$

$$u_{z,inner}^{\text{SMC}} = -m \eta_{z,inner} \tanh\left(\frac{s_z}{\psi_{z,inner}}\right),$$

¹³If our F_z command were sent directly from the outer loop it would be far too small to keep the rocket stable. Lets say the rockets' attitude θ_d was required to be at 45 degrees by our outer loop and the rocket was currently vertical, our F_z command sent to the actuator would be far too small to maintain the rockets' altitude z .

Giving

$$F_{z,inner} = F_{z,inner}^{BSC} + u_{z,inner}^{SMC}$$

3.6.2 Inner Loop Control Allocation

Here we allocate the inner loop controls to be sent to the actuators. First we collate the Sliding Mode and Backstepping commands for both the z channel and the θ channel.

From our total signal output for the attitude loop we have

$$u_\theta = M^{BSC} + u_\theta^{SMC},$$

since u_θ has units Nm we can calculate the required F_x to achieve this moment,

$$F_x = \frac{M}{L} \quad (16)$$

Now we have both our required force in the x and z direction we can allocate these to the actuators using:

$$T_{unclipped} = \sqrt{F_{x,ref}^2 + F_{z,ref}^2},$$

$$\phi_{unclipped} = \arctan 2(F_x, F_z) - \theta,$$

3.7 Saturate outputs

Now we set saturation limits to ensure that the output is feasible for the actuators to achieve, these are laid out in Table 2.

$$F_{e,final} = \text{clip}\left(\sqrt{F_x^2 + F_z^2}, F_{e,min}, F_{e,max}\right),$$

$$\phi_{final} = \text{clip}\left(\arctan 2(F_x, F_z) - \theta, -\phi_{max}, +\phi_{max}\right),$$

where

$$\text{clip}(x, a, b) = \min(\max(x, a), b).$$

Chapter Summary

This chapter derives a two-loop Backstepping–Sliding Mode controller for landing a thrust-vectoring rocket. An outer loop combines Backstepping virtual forces $\mathbf{u}_{\text{virtual}}^{\text{BSC}}$ with smooth SMC terms $\mathbf{u}_{\text{virtual}}^{\text{SMC}}$ to compute desired thrust-vector angle θ_d . An inner loop backsteps through attitude dynamics to achieve θ_d adding a Backstepping moment M^{BSC} to u_{θ}^{SMC} which then real-locates forces F_x, F_z to engine thrust F_e and gimbal φ , saturating any large outputs to within actuator limits.

4 Simulations

To validate the hybrid Backstepping–Sliding Mode controller under realistic operating conditions, we conduct a series of simulations of the 3-DoF thrust-vectoring rocket. Each run uses the trapezoidal-velocity reference trajectories from Section 4.1, driving the vehicle from $(x, z) = (0, 0)$ at $t = 0$ to $(2, 2)$ at $t = 5$ s and then to $(4, 0)$ at $t = 10$ s. We impose large, time-varying force and moment disturbances to the model, up to ± 30 N in F_x and F_z , ± 30 Nm in M . This is done to mimic wind gusts, inaccurate state estimation, propellant slosh, and modelling errors, plus a 20% mass-estimation error. By comparing demanded vs actual states and tracking errors (Figures 8 and 7), we assess position and attitude accuracy, disturbance rejection, and actuator usage, thereby demonstrating the controller’s robustness and practical viability.

4.1 Trajectory-Planning for Reference Generation

Often a controller such as the one outlined in this report will have a trajectory guidance algorithm which will give the rocket feasible states to follow. Therefore we must ensure that the controller receives kinematically and physically feasible states for controller testing, to emulate real world guidance algorithms, we generate smooth, piecewise-continuous reference trajectories in each axis using a *trapezoidal-velocity* profile. An example of this profile is given in Figure 6. We do

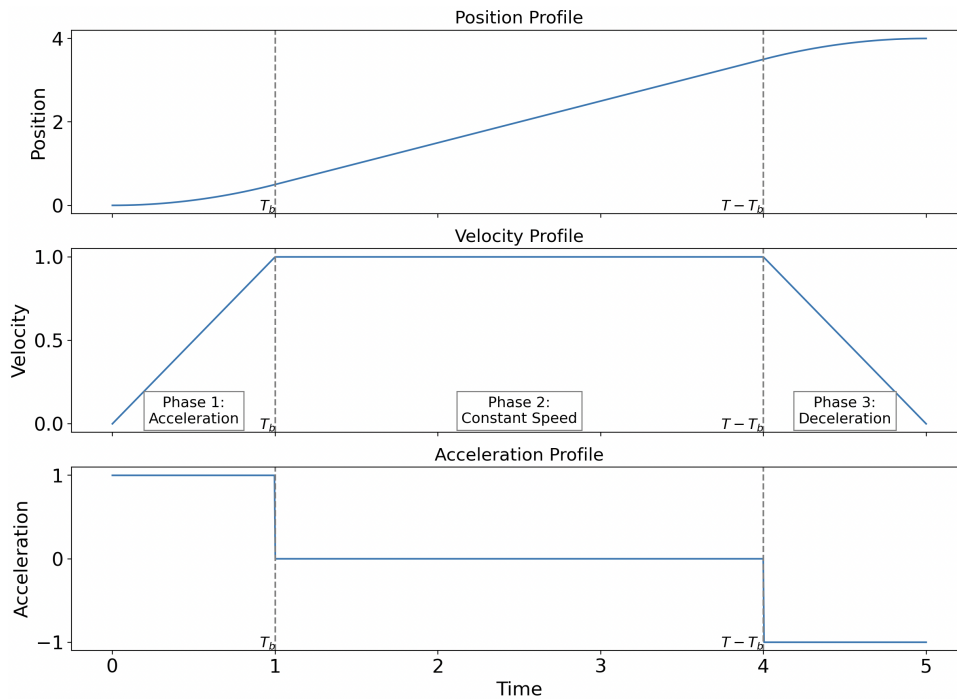


Figure 6: Example of a trapezoidal velocity profile.

this by first defining the initial (x_0, z_0) and final desired positions of the rocket (x_f, z_f) , the total manoeuvre time $T > 0$, and a blending fraction β so that each axis spends

$$T_b = \beta T$$

accelerating from zero to peak speed and another T_b decelerating back to zero.

$$T_c = T - 2T_b.$$

A maximum nominal acceleration $a_{\max}$ chosen so that the resulting peak speed does not exceed actuator capabilities. Focusing on the x -axis (identical formulas apply in z):

$$\Delta x = x_f - x_0, \quad T_b = \beta T, \quad T_c = T - 2T_b.$$

During the two parabolic phases (acceleration and deceleration), the distance covered is

$$D_{\text{acc}} = \frac{1}{2} a_x T_b^2, \quad D_{\text{dec}} = \frac{1}{2} a_x T_b^2,$$

and during the constant-speed phase,

$$D_{\text{cruise}} = V_x T_c, \quad V_x = a_x T_b.$$

Hence the total distance constraint becomes

$$\Delta x = D_{\text{acc}} + D_{\text{cruise}} + D_{\text{dec}} = a_x T_b^2 + (a_x T_b) T_c = a_x (T_b^2 + T_b T_c),$$

which yields

$$a_x = \frac{\Delta x}{T_b^2 + T_b T_c}, \quad V_x = a_x T_b.$$

Similarly, in the z -axis one finds

$$\Delta z = z_f - z_0, \quad a_z = \frac{\Delta z}{T_b^2 + T_b T_c}, \quad V_z = a_z T_b.$$

In practice, we verify that a_x, a_z do not exceed reasonable actuator limits. For example here our engine can produce a maximum of 500N and the vehicle weighs 25kg so a required a_z of

higher than $10m/s^2$ is going to be unreasonable for the system.

We therefore define our desired positions, velocities, and accelerations over time as follows:

$$\ddot{x}_d(t) = \begin{cases} 0, & t \leq 0 \\ +a_x, & 0 \leq t \leq T_b, \\ 0, & T_b < t \leq T - T_b, \\ -a_x, & T - T_b < t \leq T, \\ 0, & T \leq t \end{cases}$$

$$\dot{x}_d(t) = \begin{cases} 0, & t \leq 0 \\ a_x t, & 0 \leq t \leq T_b, \\ V_x, & T_b < t \leq T - T_b, \\ V_x - a_x (t - (T - T_b)), & T - T_b < t \leq T, \\ 0, & T \leq t \end{cases}$$

$$x_d(t) = \begin{cases} 0, & t \leq 0 \\ x_0 + \frac{1}{2} a_x t^2, & 0 \leq t \leq T_b, \\ x_0 + \frac{1}{2} a_x T_b^2 + V_x (t - T_b), & T_b < t \leq T - T_b, \\ \left[x_0 + \frac{1}{2} a_x T_b^2 + V_x T_c \right] + V_x \tau - \frac{1}{2} a_x \tau^2, & T - T_b < t \leq T, \\ 0 & T \leq t \end{cases}$$

where $\tau = t - (T - T_b)$. Analogous definitions hold for $\ddot{z}_d(t)$, $\dot{z}_d(t)$, $z_d(t)$ by replacing (x_0, x_f, a_x, V_x) with (z_0, z_f, a_z, V_z) . We then output this to make up our references for the controller to use with:

$$s_r(t) = \begin{bmatrix} x_d(t) \\ z_d(t) \end{bmatrix}, \quad \dot{s}_r(t) = \begin{bmatrix} \dot{x}_d(t) \\ \dot{z}_d(t) \end{bmatrix}, \quad \ddot{s}_r(t) = \begin{bmatrix} \ddot{x}_d(t) \\ \ddot{z}_d(t) \end{bmatrix}.$$

These six signals $\{x_d, \dot{x}_d, \ddot{x}_d, z_d, \dot{z}_d, \ddot{z}_d\}$ feed the outer-loop controller at each time step.

Blending Function

In Figure 6 there is an abrupt change in the acceleration profile at the end of the blending sections. This has the effect of causing a discontinuity in the acceleration errors ($\ddot{e}_z, \ddot{e}_x, \ddot{e}_\theta$), this then causes a discontinuity in the demanded angle θ_d which is fed into the inner loop. This sudden step change is not ideal in a real system with actuators, as it can cause excessive wear, therefore a blending function has been applied to the theoretical trajectory.

If we let $f_1(t)$ and $f_2(t)$ represent two time-varying functions (e.g. position, velocity, or acceleration) that need to be smoothly transitioned between over a time interval $t_0 \rightarrow t_0 + \Delta t$. Define the normalized blend parameter:

$$\alpha(t) = \frac{t - t_0}{\Delta t} \quad (17)$$

Then the blended function $f(t)$ is given by

$$f(t) = (1 - \sigma(\alpha))f_1(t) + \sigma(\alpha)f_2(t). \quad (18)$$

Where

$$\sigma(\alpha) = 6\alpha^5 - 15\alpha^4 + 10\alpha^3, \quad \alpha \in [0, 1] \quad (19)$$

We use a quintic polynomial so that we can ensure the following boundary conditions:

$$\begin{aligned} \sigma(0) &= 0 && \text{ensures the blend starts exactly at } f_1(t), \\ \sigma(1) &= 1 && \text{ensures the blend ends exactly at } f_2(t), \\ \sigma'(0) &= 0, \sigma'(1) = 0 && \text{ensures zero velocity at both ends,} \\ \sigma''(0) &= 0, \sigma''(1) = 0 && \text{ensures zero acceleration at both ends.} \end{aligned} \quad (20)$$

If these boundary conditions are met it means that the overall blended function smoothly transitions between $f_1(t)$ and $f_2(t)$ with respect to position, velocity, and acceleration.

4.2 Simulations

Simulations of the control system and its model validate our results from the controller design section. They are a verification step for the controller and allow us to see how the system behaves, because technically our system should be stable for any positive gain, but just because something is stable (reaching the final demanded position in finite time) it does not mean the transient behaviour is desirable (large overshoots or very sluggish behaviour).

In this simulation campaign we want to show that, with the right gains selected, the model performs well and completes its objectives.

To test this we simulated the full rocket taking off from point $(0, 0)$ at $t = 0$, flying to $(2, 2)$ at $t = 5s$ and then landing at $(4, 0)$ at $t = 10s$, using two parabolic trajectories outlined above in Section 4.1.

To ensure the control system is robust against any disturbances, large disturbances are added in the form of forces in the F_x and F_z directions and a moment in M_θ , all about the centre of mass. These have been added to simulate the often unpredictable non-linear disturbances which wind, propellant slosh, inaccurate state estimation or even actuator backlash can produce.

They are defined as follows:

$$\mathbf{d}(t) = \begin{bmatrix} d_x(t) \\ d_z(t) \\ m_x(t) \end{bmatrix} = \begin{bmatrix} 20 \sin 2t + 10 \sin (6t + \frac{\pi}{2}) \\ 20 \cos 3t \\ 20 \sin 2t \end{bmatrix}. \quad (21)$$

A 20% mismatch of the mass is also assumed, this is to simulate inaccurate state estimation as the mass cannot be directly measured when the vehicle is in flight and must be estimated from other parameters. This leads to

$$m_{model} = 1.2 m_{controller},$$

where $m_{controller}$ is the mass which the controller uses for its calculations and m_{model} is the mass used in the model. To give context to these values, on a windy day at the Silwood park rocketry test site, wind gusts can reach up to $35mph$ [14] which corresponds to $27N^{14}$.

¹⁴Here we assume drag over a cylinder with $C_D = 1$, $A = 0.18m^2$ at sea level, then use $F_D = \frac{1}{2} \rho v^2 C_D A$

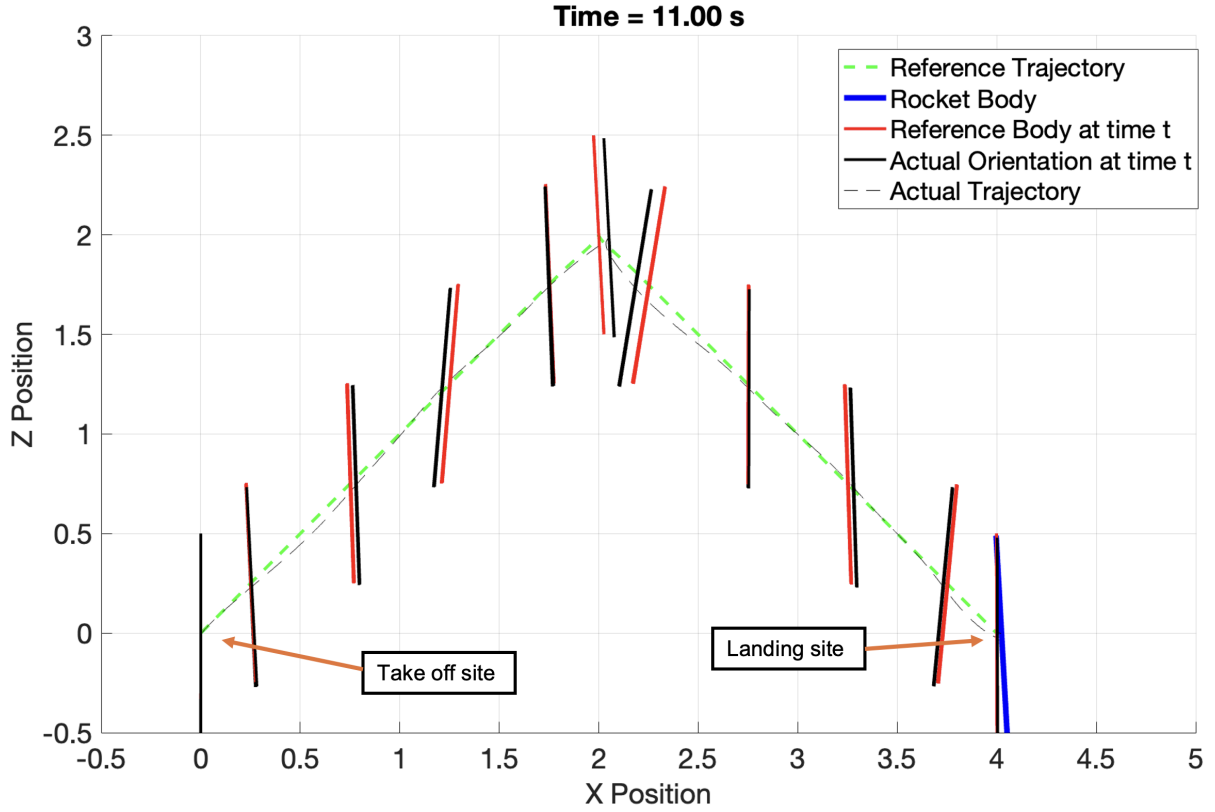


Figure 7: Full flight path of the rocket with orientation references every 1 second

Therefore our horizontal disturbance assumes a highly non-linear maximum wind gust, the worst case scenario. The added $10 \sin(6t + \frac{\pi}{2})$ is to mimic propellant sloshing from side to side in the vehicle also for further proof of the controllers robust nature. For our vertical disturbance $d_z(t)$, a similar non-linear disturbance has been added to mimic some of the issues that may arise from a delay or mismatch of thrust in the throttling of the rocket engine, often around 5% of the total engine thrust [15]. Finally, the last simulated external disturbance is a non-linear moment about the centre of mass of the rocket, which is to mimic a centre of mass that is not above the central axis of the vehicle or even moving due to propellant slosh/uneven loss, thereby creating a moment on the vehicle.¹⁵ The time-varying-disturbances are summarised in Figure 8 d. The results of the simulation are outlined in Figure 7 and the corresponding errors in Figures 8 (a)&(c).

Figure 7 captures the rocket's position and orientation every $1s$ in black and the reference at that time step in red, over the course of the $11s$ flight. In the image we can see that the orientation θ follows the reference orientation θ_d well over the course of the flight. This is fur-

¹⁵ $20Nm$ is quite a high moment for such a vehicle to experience, however, for the rigour of the controller we must examine the worst case scenario, which in this case would be the centre of the mass of the rocket being at the edge of the rocket (with added propellant), giving $M = FL = mgL = 25 * 9.81 * 0.075$

ther exemplified in Figure 8 c, which shows a close following to the reference orientation, with absolute error never exceeding 0.04 rad or 2.3°.

In Figure 7 we also see a close agreement of the vehicle's reference position to its actual position. This is further supported in Figures 8a & 8b, where the absolute error does not exceed 0.08m in the x channel and 0.03m in the z channel. The z channels' absolute error is lower than the x channels' absolute error, which can be attributed to the force F_x being responsible for controlling two states θ and x . Since the attitude controller uses F_x to command the required rotation, this results in a non-minimum phase behaviour from F_x as the controller initially has to create a negative F_x , initially increasing the x channel error, to rotate the vehicle to satisfy the attitude $\theta = \theta_d$.

4.3 Parameters

Below is a table which summarises the parameters used in the model and controller.

Group	Parameter	Value	Units	Description
<i>Initial Conditions</i>	m_0	25	kg	Initial mass
	J_0	3	kg·m ²	Initial moment of inertia
<i>Dynamic Model</i>	L_1	1	m	Engine lever arm
	α_1	2.7×10^{-4}	–	Mass-flow coefficient
	α_2	1.5×10^{-4}	–	Inertia-flow coefficient
	g	9.81	m/s ²	Gravitational acceleration
<i>Backstepping Gains</i>	k_1, k_2	20	–	Attitude inner-loop gains
	k_3	4	–	Vertical outer-loop damping
	k_4	5	–	Vertical outer-loop stiffness
	k_5	5	–	Lateral outer-loop damping
	k_6	2	–	Lateral outer-loop stiffness
	k_7, k_8	10	–	Vertical inner-loop gains
<i>Sliding-Mode Surface Gains</i>	$\lambda_x, \lambda_z, \lambda_{th}$	8	–	Surface slope gains
<i>Sliding-Mode Amplitudes</i>	η_x, η_z	0.5	–	Outer-loop SMC amplitude
	η_{th}	2	–	Inner-loop SMC amplitude
<i>Sliding-Mode Smoothing</i>	ψ_x, ψ_z	0.1	–	Outer-loop boundary thickness
	ψ_{th}	0.2	–	Inner-loop boundary thickness
<i>Actuator Limits</i>	$F_{e,max}$	500	N	Max engine thrust
	$F_{e,min}$	25	N	Min thrust (5% of max)
	ϕ_{max}	$\pm 7^\circ$	rad	Gimbal angle limit

Table 2: Summary of model and controller parameters

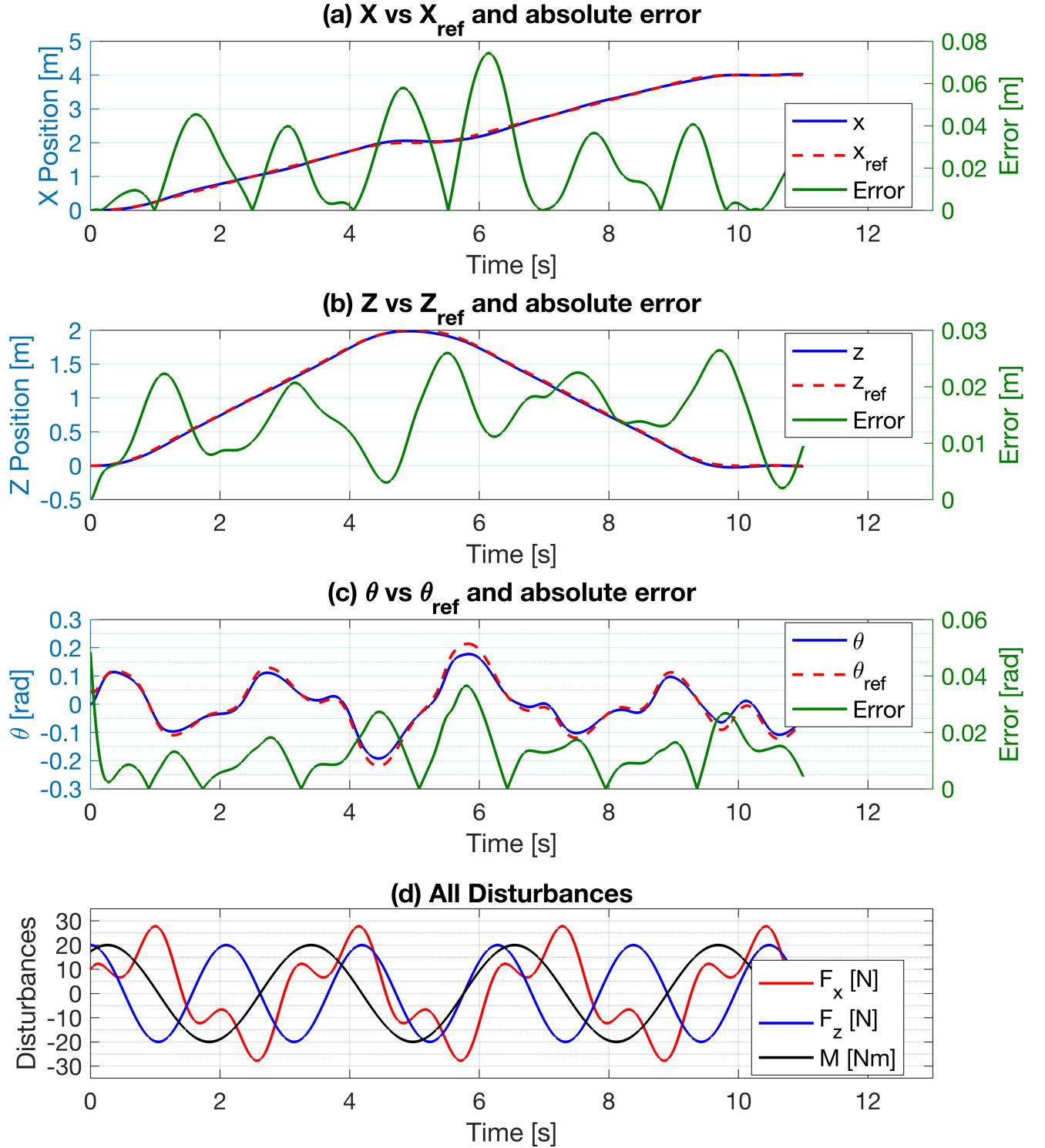


Figure 8: Demanded (a) x_{ref} (b) z_{ref} (c) θ_{ref} , (left hand vertical axis) vs actual state (a) x ,(b) z ,(c) θ during flight with their absolute errors (right hand vertical axis) and disturbances in the F_x , F_z , & M_θ (d).

5 Conclusions & Future work

In this project, we have developed, analysed, and validated a **novel** two-loop control architecture for the vertical landing of a thrust-vectoring **reusable rocket**. By combining Lyapunov-based **Backstepping** with a smooth **Sliding Mode** correction term in both the translational (outer) and attitude (inner) loops, global asymptotic stability is assured for all tracking errors for any choice of positive controller gains. The outer loop provides position control, mapping desired horizontal and vertical forces into a thrust-vector angle for the inner loop to track.

Our high-fidelity 3-DoF simulations, driven by smooth trapezoidal-velocity reference trajectories and subjected to large, time-varying disturbances (wind gusts, propellant slosh, 20 % mass uncertainty), demonstrate:

- **Accurate position and attitude tracking**, with errors converging to zero in under 1 s after each trajectory segment;
- **Robust disturbance rejection**, maintaining close agreement with the desired states despite ± 30 N/ ± 30 Nm disturbances;
- **Actuator feasibility**, keeping thrust and gimbal commands within the 25 N–500 N and $\pm 7^\circ$ bounds.

These results validate that our hybrid controller meets the dual goals of *performance* (precision and robustness) and *practicality* (computational efficiency, customisability, and tuneable gains).

Looking forwards to extension of this work in the future, this control system has proven global asymptotic stability under significant disturbances but only in a 3-DoF landing scenario. Extension of the model and controller to a 6-DoF simulation with incorporation of further high fidelity dynamics such as a non-rigid body, unlike in this model, would be beneficial for understanding this problem. Also a practical implementation of a control system onto a vehicle would provide much needed scope for where to focus future research regarding the rocket landing problem.

References

- [1] E. Mathieu, P. Rosado, and M. Roser. "Data page: Cost of space launches to low earth orbit." Data adapted from CSIS Aerospace Security Project. (2022), [Online]. Available: <https://ourworldindata.org/grapher/cost-space-launches-low-earth-orbit> (visited on 05/21/2025).
- [2] J. D. V. Smitherman, "New space industries for the next millennium," NASA Marshall Space Flight Center, Huntsville, Alabama, Tech. Rep. NASA/CP-1998-209006, Dec. 1998, Compiled by D. V. Smitherman, Jr. [Online]. Available: <https://ntrs.nasa.gov/api/citations/19990021561/downloads/19990021561.pdf>.
- [3] Space Exploration Technologies Corp., *Falcon payload user's guide*, Accessed: 2025-06-01, SpaceX, May 2025. [Online]. Available: <https://www.spacex.com/media/falcon-users-guide-2025-05-09.pdf>.
- [4] NASASpaceflight, *SpaceX Launches Starship Flight 5 (and Catches A Booster)*, YouTube video, Oct. 2024. [Online]. Available: https://www.youtube.com/watch?v=YC87WmFN_As.
- [5] H. Dave, S. Saxena, and J. Mukherjee, "Integral state feedback control based vertical landing of reusable rockets," in *Proceedings of the 10th International Conference on Control, Decision and Information Technologies (CoDIT)*, Valletta, Malta, Jul. 2024, pp. 1111–1116. [Online]. Available: <https://doi.org/10.1109/CoDIT62066.2024.10708402>.
- [6] A. Ganbold, "Design of a linear quadratic gaussian control system for a thrust vector controlled rocket," Master's Thesis, San José State University, May 2023. [Online]. Available: <https://www.sjsu.edu/ae/docs/project-thesis/Alex.Ganbold-Su23.pdf>.
- [7] P. d. Santos and P. Oliveira, "Integrated architecture for navigation and attitude control of low-cost suborbital launch vehicles," *Acta Astronautica*, vol. 222, pp. 52–68, Sep. 2024, issn: 0094-5765. doi: 10.1016/j.actaastro.2024.05.039. [Online]. Available: <http://dx.doi.org/10.1016/j.actaastro.2024.05.039>.
- [8] K.-W. Jung, S.-D. Lee, C.-G. Jung, and C.-H. Lee, *Model predictive guidance for fuel-optimal landing of reusable launch vehicles*, 2024. arXiv: 2405.01264 [eess.SY]. [Online]. Available: <https://arxiv.org/abs/2405.01264>.

- [9] Z. Xia, Z. Xu, Y. Liu, L. Zhang, and M. Li, "Adaptive trajectory tracking control of a reusable launch vehicle considering input saturation," *Aerospace*, vol. 12, no. 2, pp. 1–17, 2024. [Online]. Available: <https://doi.org/10.3390/aerospace12020111>.
- [10] F. Wang, Q. Zong, and B. Tian, "Adaptive backstepping finite time attitude control of reentry rlv with input constraint," *Mathematical Problems in Engineering*, vol. 2014, no. 1, p. 801747, 2014. doi: <https://doi.org/10.1155/2014/801747>.
- [11] L. Ye, J. Li, C. Wang, H. Liu, and B. Liang, "Control of an underactuated reusable launch vehicle by partial feedback linearization," Jun. 2021. doi: 10.23919/CCC52363.2021.9549314.
- [12] Y. B. Shtessel, J. J. Zhu, and D. Daniels, "Reusable Launch Vehicle Attitude Control Using a Time-Varying Sliding Mode Control Technique," University of Alabama in Huntsville, Huntsville, AL, Tech. Rep. NASA/CR-20020092011, 2002. [Online]. Available: <https://ntrs.nasa.gov/api/citations/20020092011/downloads/20020092011.pdf>.
- [13] R. Mallik, *Robust control approaches for minimizing the bandwidth ratio in multi-loop control*, 2022. eprint: 2203.09022. [Online]. Available: <https://arxiv.org/abs/2203.09022>.
- [14] "Top hat and gales! ascot racegoers hit by 35mph winds... and there will be gale-force gusts tomorrow." Published June22,2013. (Jun. 2013), [Online]. Available: <https://www.dailymail.co.uk/news/article-2346401/Top-hat-gales-Ascot-racegoers-hit-35mph-winds--gale-force-gusts-tomorrow.html> (visited on 06/11/2025).
- [15] Imperial College London. "Imperial student rocketry team wins international challenge." Accessed: 2025-06-11. (2024), [Online]. Available: <https://www.imperial.ac.uk/news/256154/imperial-student-rocketry-team-international-challenge/>.

Appendices

A Full Mechanics Derivation

A.1 Rotation Matrices

We denote by $R_{B \rightarrow G}$ the rotation matrix that *maps a vector expressed in body-fixed axes into its components in global axes*, and by $R_{G \rightarrow B} = R_{B \rightarrow G}^T$ the inverse mapping (global→body). Below, we present explicit forms in 2D and in 3D.

When motion is restricted to a plane (e.g. the X – Y plane of the global frame), the body frame B can be related to G by a single angle θ . Let:

$$\theta = \text{angle between } \mathbf{e}_X \text{ (global } X) \text{ and } \mathbf{e}_x \text{ (body } x).$$

By convention, a positive θ corresponds to a counterclockwise rotation of B relative to G . Then any 2D vector $\mathbf{v}$ has components ${}^B\mathbf{v} = [v_x, v_y]^T$ in the body frame and ${}^G\mathbf{v} = [v_X, v_Y]^T$ in the global frame. The rotation matrix $R_{B \rightarrow G}$ is

$$R_{B \rightarrow G}^{(2D)}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad (22)$$

so that

$${}^G\mathbf{v} = R_{B \rightarrow G}^{(2D)}(\theta) {}^B\mathbf{v} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix}.$$

The inverse mapping (global→body) is simply

$$R_{G \rightarrow B}^{(2D)}(\theta) = (R_{B \rightarrow G}^{(2D)}(\theta))^T = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}, \quad (23)$$

since $[R(\theta)]^{-1} = R(-\theta) = R(\theta)^T$.

A.2 Linear Acceleration

Starting directly with Newton's second law in the inertial (global) frame G :

$$m \, {}^G\dot{\mathbf{v}}_{\text{CoM}}^G = {}^G\mathbf{F}. \quad (1)$$

Where:

- ${}^G\dot{\mathbf{v}}_{\text{CoM}}^G$ denotes the CoM acceleration of the body *relative to* G , expressed in the global axes.
- ${}^G\mathbf{F}$ is the total external force (thrust, drag, etc.) expressed in G .
- m is the instantaneous mass of the rocket.

In the model, the external force (${}^B\mathbf{F}$) is expressed in the body axes. It is related to the global force by the rotation matrix $R_{B \rightarrow G}$:

$${}^G\mathbf{F} = R_{B \rightarrow G} {}^B\mathbf{F}. \quad (24)$$

Substituting (2) into (1) and dividing by m yields the complete inertial-frame acceleration in a single line:

$$\boxed{{}^G\dot{\mathbf{v}}_{\text{CoM}}^G = \frac{1}{m} {}^G\mathbf{F}.}$$

No Coriolis or gyroscopic terms appear, those only arise when differentiating vectors in the rotating body frame. If a body-frame acceleration is required, one can transform 2 back into B and apply the transport theorem to obtain the following expression

$${}^B\dot{\mathbf{v}}_{\text{CoM}}^G = \frac{1}{m} {}^B\mathbf{F} - ({}^B\boldsymbol{\omega}_B^G \times {}^B\mathbf{v}_{\text{CoM}}^G)$$

Where,

- ${}^B\dot{\mathbf{v}}_{\text{CoM}}^G$ denotes the CoM acceleration *relative to* G , expressed in body-fixed axes.
- ${}^B\mathbf{v}_{\text{CoM}}^G$ is the corresponding velocity.
- ${}^B\mathbf{F}$ is the total external force (thrust, drag, etc.) expressed in B .
- m is the instantaneous mass of the rocket.
- ${}^B\boldsymbol{\omega}_B^G$ is the angular velocity of frame B relative to G , expressed in B .

A.3 Angular Acceleration

We begin with the rotational dynamics in the non-inertial body frame B . In this frame:

- ${}^B\boldsymbol{\omega}_B^G$ is the angular velocity of B relative to the inertial frame G , expressed in body-fixed axes.
- ${}^B\dot{\boldsymbol{\omega}}_B^G$ is the angular acceleration of B relative to G , expressed in B .
- ${}^B\mathbf{M}$ is the total external moment (torque) about the center of mass, expressed in B .
- J_B is the inertia tensor of the rocket about its CoM, expressed in B .

The body-frame centric Euler's equation for a rigid body is,

$$J_B {}^B\dot{\boldsymbol{\omega}}_B^G + {}^B\boldsymbol{\omega}_B^G \times (J_B {}^B\boldsymbol{\omega}_B^G) = {}^B\mathbf{M}. \quad (25)$$

Rearranging (25) to solve explicitly for the body-frame angular acceleration gives

$${}^B\dot{\boldsymbol{\omega}}_B^G = J_B^{-1} \left({}^B\mathbf{M} - {}^B\boldsymbol{\omega}_B^G \times (J_B {}^B\boldsymbol{\omega}_B^G) \right). \quad (26)$$

Next, to obtain the angular acceleration expressed in the global frame G , we note that ${}^B\boldsymbol{\omega}_B^G$ is a vector whose components are given in B . For any vector ${}^B\mathbf{x}$ (here $\mathbf{x} = \boldsymbol{\omega}_B^G$), its time derivative in G obeys the transport theorem:

$${}^G\frac{d}{dt}({}^B\mathbf{x}) = R_{B \rightarrow G} \left({}^B\dot{\mathbf{x}} + {}^B\boldsymbol{\omega}_B^G \times {}^B\mathbf{x} \right), \quad (27)$$

where $R_{B \rightarrow G}$ is the rotation matrix from body-fixed axes to global (inertial) axes.

Applying (27) with $\mathbf{x} = \boldsymbol{\omega}_B^G$ yields

$${}^G\dot{\boldsymbol{\omega}}_B^G = {}^G\frac{d}{dt}({}^B\boldsymbol{\omega}_B^G) = R_{B \rightarrow G} \left({}^B\dot{\boldsymbol{\omega}}_B^G + {}^B\boldsymbol{\omega}_B^G \times {}^B\boldsymbol{\omega}_B^G \right). \quad (28)$$

Since ${}^B\boldsymbol{\omega}_B^G \times {}^B\boldsymbol{\omega}_B^G = \mathbf{0}$, equation (28) simplifies to

$${}^G\dot{\boldsymbol{\omega}}_B^G = R_{B \rightarrow G} {}^B\dot{\boldsymbol{\omega}}_B^G. \quad (29)$$

Substitute (26) into (29):

$${}^G\dot{\boldsymbol{\omega}}_B^G = R_{B \rightarrow G} J_B^{-1} \left({}^B\mathbf{M} - {}^B\boldsymbol{\omega}_B^G \times (J_B {}^B\boldsymbol{\omega}_B^G) \right).$$

Define the global-frame inertia tensor and moment as

$$J_G := R_{B \rightarrow G} J_B R_{B \rightarrow G}^T,$$

$${}^G\mathbf{M} := R_{B \rightarrow G} {}^B\mathbf{M}.$$

Also note that

$${}^G\boldsymbol{\omega}_B^G = R_{B \rightarrow G} {}^B\boldsymbol{\omega}_B^G,$$

and therefore

$$J_G {}^G\boldsymbol{\omega}_B^G = R_{B \rightarrow G} (J_B {}^B\boldsymbol{\omega}_B^G).$$

Putting these definitions into the previous expression yields the angular acceleration in the global frame:

$$\boxed{{}^G\dot{\boldsymbol{\omega}}_B^G = J_G^{-1} \left({}^G\mathbf{M} - {}^G\boldsymbol{\omega}_B^G \times (J_G {}^G\boldsymbol{\omega}_B^G) \right)}. \quad (30)$$

A.4 Input matrices

$$R_{B \rightarrow G}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}, \quad {}^B\mathbf{F}_{\text{thrust}} = \begin{bmatrix} T \sin \varphi \\ T \cos \varphi \end{bmatrix}, \quad {}^G\mathbf{F}_{\text{grav}} = \begin{bmatrix} 0 \\ -mg \end{bmatrix}.$$

$${}^B\mathbf{r} = \begin{bmatrix} L \\ 0 \\ 0 \end{bmatrix}, \quad {}^B\mathbf{F} = \begin{bmatrix} T \sin \varphi \\ T \cos \varphi \\ 0 \end{bmatrix}, \quad {}^B\mathbf{M} = \begin{bmatrix} 0 \\ 0 \\ LT \sin \varphi \end{bmatrix}, \quad J_B = J.$$

B α_1 & α_2 derivation

Mass-flow coefficient α_1

The thrust produced by a rocket nozzle can be written in terms of the exhaust mass-flow and velocity:

$$T = \dot{m}_e v_e + (p_e - p_a) A_e \approx \dot{m}_e v_e \quad (\text{neglecting pressure term by assuming perfect expansion}).$$

By sign convention, the rate of change of onboard propellant mass is

$$\dot{m} = -\dot{m}_e, \quad \dot{m} < 0,$$

so that

$$T = -\dot{m} v_e.$$

Defining the *effective exhaust velocity* $c \equiv v_e$, we have

$$T = -\dot{m} c \Rightarrow \dot{m} = -\frac{T}{c}.$$

Since $c = I_{\text{sp}} g_0$ where $c = I_{\text{sp}} g_0$ is the specific impulse of your engine, with $g_0 \approx 9.80665 \text{ m/s}^2$, it follows that

$$\dot{m} = -\frac{T}{I_{\text{sp}} g_0} \Rightarrow \alpha_1 = \frac{1}{I_{\text{sp}} g_0}.$$

Moment-of-inertia coefficient: α_2

For a solid cylinder of radius R , the moment of inertia about its central axis is

$$J = \frac{1}{2} m R^2.$$

Differentiating with respect to time and using the mass-flow law gives

$$\dot{J} = \frac{d}{dt} \left(\frac{1}{2} m R^2 \right) = \frac{1}{2} R^2 \dot{m} = \frac{1}{2} R^2 \left(-\frac{T}{c} \right) = -\frac{R^2}{2c} T.$$

Hence

$$\alpha_2 = \frac{R^2}{2c} = \frac{R^2}{2I_{\text{sp}} g_0}.$$

C Lyapunov Function and Stability Intuition

A Lyapunov function $V(x_1, x_2)$ can be thought of as a smooth, bowl-shaped surface over the two-dimensional state space (x_1, x_2) , as illustrated in Figure 9: its vertical axis is the scalar value V , which is zero at the origin and strictly positive elsewhere. When we design a control law so that the time-derivative $\dot{V} = \nabla V \cdot \dot{x}$ is negative (or negative definite) everywhere except at the equilibrium, trajectories are forced to “roll downhill” toward the bowl’s minimum at $(0, 0)$. In this way, the choice of V and the condition $\dot{V} < 0$ guarantee that the state converges asymptotically to the desired setpoint without solving the system dynamics explicitly.

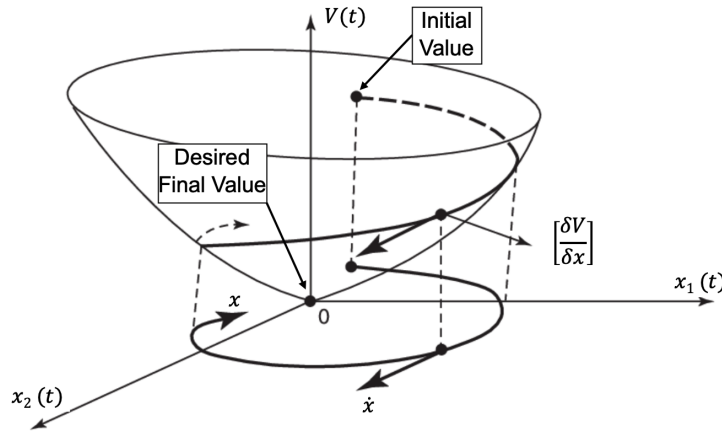


Figure 9: Schematic of a Lyapunov function $V(x_1, x_2)$ as a bowl over the state space.

When forming a Lyapunov function we typically choose

$$V = \underbrace{\frac{1}{2} e^2}_{\text{potential energy}} + \underbrace{\frac{1}{2} m s^2}_{\text{kinetic energy}} \quad (\text{or} \quad \underbrace{\frac{1}{2} J s^2}_{\text{rotational kinetic energy}}).$$

so that each term represents a form of physical energy (potential¹⁶ or kinetic¹⁷) and V is positive definite with consistent units. By ensuring that the derivative of the energy of the system is always negative with time we can ensure that the system will tend towards zero energy with time, therefore making the system stable.

¹⁶This term mimics the potential energy stored in a virtual linear spring, $V_{\text{spring}} = \frac{1}{2} k x^2$ with $k = 1$.

¹⁷This term represents kinetic energy, $T = \frac{1}{2} m v^2$, where s plays the role of a velocity residual.

Raw Matlab Code

```
1
2 function [s_ddot,mdot,Jdot, d] = rocketDynamics(s,u,m,J,t)
3
4 %initialise states
5 L1 = 1;
6 alpha1 = 0.009;
7 alpha2 = 0.005;
8 beta = 0.03;
9 g = 9.81;
10
11 %extract states
12 Fe = u(1);
13 phi = u(2);
14 theta = s(3);
15
16 %find mdot and Jdot
17 mdot = -alpha1*(beta*Fe);
18 Jdot = -alpha2*(beta*Fe);
19
20 % Disturbances
21 disturbances = 1;
22 if disturbances == 1
23     Fx_dist = 20*sin(2*t)+10*sin(6*t+pi/2);    %N
24     Fz_dist = 20*cos(3*t);    %N
25     M_dist = 20*sin(2*t+pi/3);    %(Nm)
26     % Mass disturbance
27     m_actual = 1.2*m;
28 else
29     Fx_dist = 0;    %N
30     Fz_dist = 0;    %N
31     M_dist = 0;    %(Nm)
32     % Mass disturbance
33     m_actual = m;
```

```

34 end
35
36 %accelerations
37 x_ddot      = (Fe*sin(theta + phi) + Fx_dist) / m_actual;
38 z_ddot      = -g + (Fe*cos(theta + phi) + Fz_dist) / m_actual;
39 theta_ddot  = (Fe*sin(phi)*L1 + M_dist) / J;
40
41 s_ddot = [x_ddot;z_ddot;theta_ddot];
42 d = [Fx_dist; Fz_dist; M_dist];
43
44 end
45 function [u, phi_fb,theta_d_dot, theta_d] = BacksteprocketControllerSMC(s,
    s_dot, s_r, s_r_dot, s_r_ddot, m, J)
46 % BacksteprocketControllerSMC  Combined Backstepping + -SlidingMode
    controller
47 %
48 %   Inputs:
49 %       s -          3×1 state vector [ x;   z;   theta ]
50 %       s_dot -      3×1 -statederivative [ x_dot; z_dot; theta_dot ]
51 %       s_r -        2×1 -referenceposition [ x_d;   z_d ]
52 %       s_r_dot -    2×1 -referencevelocity [ x_d_dot; z_d_dot ]
53 %       s_r_ddot -   2×1 -referenceaccel    [ x_d_ddot; z_d_ddot ]
54 %       m -          (instantaneous) mass
55 %       J -          (instantaneous) inertia
56 %
57 %   Outputs:
58 %       u -          2×1 vector of actuator commands [ Fe_cmd;   phi_cmd ]
59 %       phi_fb - same as phi_cmd (for logging/feedback)
60 %
61 %   Implements the pseudocode for a -two-loop Backstepping + SMC controller.
62
63 %--- 1. Constants / Gains ---
64 persistent g L ...
65           k1 k2 k3 k4 k5 k6 k7 k8...
66           lambda_x lambda_z lambda_th ...
67           eta_x eta_z eta_th ...

```

```

68         psi_x psi_z psi_th ...
69         Fe_min Fe_max phi_max
70 if isempty(g)
71     g          = 9.81;           % gravitational acceleration
72     L          = 1.0;           % engine lever arm [m]
73     %gains
74     k1 = 20;  k2 = 20; % attitude
75     k3 = 4.0;  k4 = 5;  % vertical outer loop
76     k5 = 5.0;  k6 = 2;  % lateral
77     k7 = 10;  k8 = 10;  % vertical inner loop
78     % -Slidingmode surface gains (positive)
79     lambda_x   = 8.0;
80     lambda_z   = 8.0;
81     lambda_th  = 8.0;
82     % -Slidingmode amplitudes (positive)
83     eta_x      = 0.5;
84     eta_z      = 0.5;
85     eta_th     = 2;
86     % -Slidingmode smoothing (positive)
87     psi_x      = 0.1;
88     psi_z      = 0.1;
89     psi_th     = 0.2;
90     % Actuator limits
91     % k1 = 4.0;  k2 = 8.0; % attitude
92     % k3 = 4.0;  k4 = 5;  % vertical outer loop
93     % k5 = 4.0;  k6 = 5;  % lateral
94     % k7 = 10;  k8 = 10;  % vertical inner loop
95     % % -Slidingmode surface gains (positive)
96     % lambda_x   = 8.0;
97     % lambda_z   = 8.0;
98     % lambda_th  = 8.0;
99     % % -Slidingmode amplitudes (positive)
100    % eta_x      = 0.5;
101    % eta_z      = 0.5;
102    % eta_th     = 0.2;
103    % % -Slidingmode smoothing (positive)

```

```

104     % psi_x    = 0.1;
105     % psi_z    = 0.1;
106     % psi_th   = 0.05;
107     % % Actuator limits
108     Fe_max_Thrust = 500; % maximum thrust in Newtons
109     Fe_min  = 0.05*Fe_max_Thrust; % minimum thrust (fraction of max)
110     Fe_max  = 1.00*Fe_max_Thrust; % maximum thrust (fraction)
111     phi_max = 7.0 * pi/180; % gimbal limit [rad]
112 end
113
114 %--- 2. Unpack states and references ---
115 x      = s(1);
116 z      = s(2);
117 theta  = s(3);
118
119 x_dot  = s_dot(1);
120 z_dot  = s_dot(2);
121 theta_dot = s_dot(3);
122
123 x_d    = s_r(1);
124 z_d    = s_r(2);
125
126 x_d_dot = s_r_dot(1);
127 z_d_dot = s_r_dot(2);
128
129 x_d_ddot = s_r_ddot(1);
130 z_d_ddot = s_r_ddot(2);
131 %--- 2. Compute -positionerrors and sliding surfaces ---
132 e_x     = x      - x_d;
133 e_x_dot = x_dot  - x_d_dot;
134
135 s_x     = e_x_dot + lambda_x * e_x;
136
137 e_z     = z      - z_d;
138 e_z_dot = z_dot  - z_d_dot;
139

```



```

140 s_z      = e_z_dot + lambda_z * e_z;
141
142 %--- 3. ""Bare backstepping references (outer loop) ---
143 Fz_ref_BS = m * ( g + z_d_ddot ...
144                 -(k3 + k4) * e_z_dot ...
145                 - k3 * k4 * e_z);
146
147 Fx_ref_BS = m * ( x_d_ddot ...
148                 - (k5 + k6) * e_x_dot ...
149                 - k5 * k6 * e_x);
150
151 %--- 4. Add -slidingmode corrections (outer loop Forces) ---
152 u_SMC_x = - m* eta_x * tanh( s_x / psi_x );
153 u_SMC_z = - m* eta_z * tanh( s_z / psi_z );
154
155 Fz_ref = Fz_ref_BS + u_SMC_z - e_z;
156 Fx_ref = Fx_ref_BS + u_SMC_x - e_x;
157
158 %--- 5. Desired attitude for -thrustvector alignment ---
159 epsilon = 10;
160 theta_d = atan2(Fx_ref, max(Fz_ref, epsilon)); %this is to ensure that
        there are no division by zero errors
161
162
163 %--- in your controller, keep track of last theta_d and last time dt ---
164 persistent theta_d_prev theta_d_dot_prev
165 if isempty(theta_d_prev)
166     theta_d_prev = theta_d;
167     theta_d_dot_prev = 0;
168 end
169
170 % current time step
171 dt = 0.0001;
172 alpha = 0.8; % pick between 0 (heavy smoothing) and 1 (no smoothing)
173 theta_d_dot_raw = (theta_d - theta_d_prev) / dt;
174 theta_d_dot = alpha*theta_d_dot_prev + (1-alpha)*theta_d_dot_raw;

```

```

175     theta_d_dot_prev = theta_d_dot;
176     theta_d_prev     = theta_d;
177
178     % pick your max angular rate (rad/s)
179     theta_d_dot_max = 60 * pi/180;    % = 1.047 rad/s
180
181     % after computing theta_d_dot_raw or filtered theta_d_dot:
182     theta_d_dot = min( max(theta_d_dot, -theta_d_dot_max), +theta_d_dot_max
183                        );
184     %here we assume there is no acceleration in theta_d
185     theta_d_ddot = 0;
186
187     %--- 6. -Attitudeerrors and sliding surface (inner loop) ---
188     e_th      = theta      - theta_d;
189     e_th_dot  = theta_dot - theta_d_dot;
190     s_th      = e_th_dot + lambda_th * e_th;
191
192     %--- 8. ""Bare backstepping moment and -verticalforce commands ---
193     M_BS = J*( theta_d_ddot ...
194               - (k1+k2)*e_th_dot ...
195               - k1*k2*e_th)...
196               - e_th;
197
198     %--- 9. Add -slidingmode corrections to -innerloop controls ---
199     u_SMC_th = - J*eta_th * tanh( s_th / psi_th );
200
201     M_ref    = M_BS    + u_SMC_th;
202
203     %--- 10. Map corrected moment to lateral force ---
204     Fx_cmd = M_ref / L;
205
206     %--- 11. Final vertical force command recalculated for the inner loop (
207             before clipping) ---
208     Fz_ref_BS = m * ( g + z_d_ddot ...
209                     -(k7 + k8) * e_z_dot ...
210                     - k7 * k8 * e_z);
211
212     u_SMC_z = - m* eta_z * tanh( s_z / psi_z );
213
214     Fz_cmd = Fz_ref_BS + u_SMC_z - e_z;
215
216     %--- 12. Convert force pair to -enginecommands ---

```

```

209 Fe_unclipped = sqrt( Fx_cmd^2 + Fz_cmd^2 );
210 phi_unclipped = atan2( Fx_cmd, Fz_cmd ) - theta;
211
212 %--- 13. Saturate engine and gimbal ---
213 Fe_cmd = min( max(Fe_unclipped, Fe_min), Fe_max );
214 phi_cmd = min( max(phi_unclipped, -phi_max), +phi_max );
215
216 %--- 14. Output actuator commands ---
217 u      = [ Fe_cmd; phi_cmd ];
218 phi_fb = phi_cmd; % for logging or feedback
219
220 %--- (15) End of control tick ---
221 end
222 %uncomment this block if you want a trapezoidal reference
223 function [s_r, sdot_r, sddot_r] = trapezoidalRefBlockXZ(t)
224     %#codegen
225     % Start, via, and final positions:
226     x0 = 0;   xi = 3;   xf = 5;
227     z0 = 0;   zi = 0;   zf = 0;
228
229     % Durations:
230     t1 = 5;%travel time for trajectory one
231     t2 = 5;%travel time for second trajectory
232
233     % Blend duration (for phase and stage transitions)
234     dt_blend = 1; % seconds
235
236     % Pre-allocate
237     s_r      = zeros(2,1);
238     sdot_r   = zeros(2,1);
239     sddot_r  = zeros(2,1);
240
241
242     % Handle after end with final stop blending
243     if t >= (t1 + t2 - dt_blend/2)
244         t_end = t1 + t2;

```

```

245     alpha = min((t - (t_end - dt_blend/2)) / dt_blend, 1);
246     sigma = 6*alpha^5 - 15*alpha^4 + 10*alpha^3;
247
248     % Evaluate trajectory at current time t
249     tau2 = t - t1;
250     tf2 = t2;
251     tb2 = 0.2 * tf2;
252     if tb2 > tf2/2
253         tb2 = tf2/2;
254     end
255     tc2 = tf2 - 2*tb2;
256     Dx2 = xf - xi;
257     Dz2 = zf - zi;
258     a_x2 = Dx2 / (tb2^2 + tb2*tc2);
259     a_z2 = Dz2 / (tb2^2 + tb2*tc2);
260     Vx2 = a_x2 * tb2;
261     Vz2 = a_z2 * tb2;
262     [s_pre, sdot_pre, sddot_pre] = trapezoidalSegment(tau2, xi, xf, zi, zf,
        a_x2, a_z2, Vx2, Vz2, tb2, tc2, dt_blend);
263
264     % Final stop values (constant position, zero velocity/accel)
265     s_post = [xf; zf];
266     sdot_post = [0; 0];
267     sddot_post = [0; 0];
268
269     s_r      = (1 - sigma)*s_pre      + sigma*s_post;
270     sdot_r   = (1 - sigma)*sdot_pre   + sigma*sdot_post;
271     sddot_r  = (1 - sigma)*sddot_pre  + sigma*sddot_post;
272     return
273 end
274
275 % Handle blending between trajectories
276 if t > t1 - dt_blend/2 && t < t1 + dt_blend/2
277     alpha = (t - (t1 - dt_blend/2)) / dt_blend;
278     sigma = 6*alpha^5 - 15*alpha^4 + 10*alpha^3;
279

```

```

280 % Stage 1 values
281 tau1 = t;
282 tf1 = t1;
283 tb1 = 0.2 * tf1;
284 if tb1 > tf1/2
285     tb1 = tf1/2;
286 end
287 tc1 = tf1 - 2*tb1;
288 Dx1 = xi - x0;
289 Dz1 = zi - z0;
290 a_x1 = Dx1 / (tb1^2 + tb1*tc1);
291 a_z1 = Dz1 / (tb1^2 + tb1*tc1);
292 Vx1 = a_x1 * tb1;
293 Vz1 = a_z1 * tb1;
294 [s1, sdot1, sddot1] = trapezoidalSegment(tau1, x0, xi, z0, zi, a_x1,
      a_z1, Vx1, Vz1, tb1, tc1, dt_blend);
295
296 % Stage 2 values
297 tau2 = t - t1;
298 tf2 = t2;
299 tb2 = 0.2 * tf2;
300 if tb2 > tf2/2
301     tb2 = tf2/2;
302 end
303 tc2 = tf2 - 2*tb2;
304 Dx2 = xf - xi;
305 Dz2 = zf - zi;
306 a_x2 = Dx2 / (tb2^2 + tb2*tc2);
307 a_z2 = Dz2 / (tb2^2 + tb2*tc2);
308 Vx2 = a_x2 * tb2;
309 Vz2 = a_z2 * tb2;
310 [s2, sdot2, sddot2] = trapezoidalSegment(tau2, xi, xf, zi, zf, a_x2,
      a_z2, Vx2, Vz2, tb2, tc2, dt_blend);
311
312 % Blend outputs
313 s_r      = (1 - sigma)*s1 + sigma*s2;

```

```

314     sdot_r = (1 - sigma)*sdot1 + sigma*sdot2;
315     sddot_r = (1 - sigma)*sddot1 + sigma*sddot2;
316     return
317 end
318
319 % Determine which stage and local time tau
320 if t <= t1
321     tau = t;
322     xA = x0;  xB = xi;
323     zA = z0;  zB = zi;
324     tf = t1;
325 else
326     tau = t - t1;
327     xA = xi;  xB = xf;
328     zA = zi;  zB = zf;
329     tf = t2;
330 end
331
332 % Trapezoidal profile setup
333 tb = 0.2 * tf;
334 if tb > tf/2
335     tb = tf/2;
336 end
337 tc = tf - 2*tb;
338
339 % Motion parameters
340 Dx = xB - xA;
341 Dz = zB - zA;
342 a_x = Dx / (tb^2 + tb*tc);
343 a_z = Dz / (tb^2 + tb*tc);
344 Vx = a_x * tb;
345 Vz = a_z * tb;
346
347 [s_r, sdot_r, sddot_r] = trapezoidalSegment(tau, xA, xB, zA, zB, a_x, a_z
    , Vx, Vz, tb, tc, dt_blend);
348 end

```

```

349
350 function [s_r, sdot_r, sddot_r] = trapezoidalSegment(tau, xA, xB, zA, zB,
    a_x, a_z, Vx, Vz, tb, tc, dt_blend)
351 s_r      = zeros(2,1);
352 sdot_r   = zeros(2,1);
353 sddot_r  = zeros(2,1);
354
355 if tau <= tb - dt_blend/2
356     sddot_r(1) = a_x;
357     sdot_r(1)  = a_x * tau;
358     s_r(1)     = xA + 0.5 * a_x * tau^2;
359
360     sddot_r(2) = a_z;
361     sdot_r(2)  = a_z * tau;
362     s_r(2)     = zA + 0.5 * a_z * tau^2;
363
364 elseif tau <= tb + dt_blend/2
365     tau_blend = tau - (tb - dt_blend/2);
366     alpha = tau_blend / dt_blend;
367     sigma = 6*alpha^5 - 15*alpha^4 + 10*alpha^3;
368
369     sddot_a_x = a_x;
370     sdot_a_x  = a_x * tau;
371     s_a_x     = xA + 0.5 * a_x * tau^2;
372
373     sddot_a_z = a_z;
374     sdot_a_z  = a_z * tau;
375     s_a_z     = zA + 0.5 * a_z * tau^2;
376
377     sddot_c_x = 0;
378     sdot_c_x  = Vx;
379     s_c_x     = xA + 0.5*a_x*tb^2 + Vx*(tau - tb);
380
381     sddot_c_z = 0;
382     sdot_c_z  = Vz;
383     s_c_z     = zA + 0.5*a_z*tb^2 + Vz*(tau - tb);

```

```

384
385     s_r(1)      = (1 - sigma)*s_a_x      + sigma*s_c_x;
386     sdot_r(1)   = (1 - sigma)*sdot_a_x   + sigma*sdot_c_x;
387     sddot_r(1)  = (1 - sigma)*sddot_a_x  + sigma*sddot_c_x;
388
389     s_r(2)      = (1 - sigma)*s_a_z      + sigma*s_c_z;
390     sdot_r(2)   = (1 - sigma)*sdot_a_z   + sigma*sdot_c_z;
391     sddot_r(2)  = (1 - sigma)*sddot_a_z  + sigma*sddot_c_z;
392
393     elseif tau <= tb + tc - dt_blend/2
394         sddot_r(1) = 0;
395         sdot_r(1)  = Vx;
396         s_r(1)     = xA + 0.5*a_x*tb^2 + Vx*(tau - tb);
397
398         sddot_r(2) = 0;
399         sdot_r(2)  = Vz;
400         s_r(2)     = zA + 0.5*a_z*tb^2 + Vz*(tau - tb);
401
402     elseif tau <= tb + tc + dt_blend/2
403         tau_blend = tau - (tb + tc - dt_blend/2);
404         alpha = tau_blend / dt_blend;
405         sigma = 6*alpha^5 - 15*alpha^4 + 10*alpha^3;
406
407         sddot_c_x = 0;
408         sdot_c_x  = Vx;
409         s_c_x     = xA + 0.5*a_x*tb^2 + Vx*(tau - tb);
410
411         sddot_c_z = 0;
412         sdot_c_z  = Vz;
413         s_c_z     = zA + 0.5*a_z*tb^2 + Vz*(tau - tb);
414
415         tau2 = tau - (tb + tc);
416         x3 = xA + 0.5*a_x*tb^2 + Vx*tc;
417         z3 = zA + 0.5*a_z*tb^2 + Vz*tc;
418
419         sddot_d_x = -a_x;

```



```

420     sdot_d_x = Vx - a_x * tau2;
421     s_d_x = x3 + Vx * tau2 - 0.5*a_x * tau2^2;
422
423     sddot_d_z = -a_z;
424     sdot_d_z = Vz - a_z * tau2;
425     s_d_z = z3 + Vz * tau2 - 0.5*a_z * tau2^2;
426
427     s_r(1)      = (1 - sigma)*s_c_x      + sigma*s_d_x;
428     sdot_r(1)   = (1 - sigma)*sdot_c_x   + sigma*sdot_d_x;
429     sddot_r(1)  = (1 - sigma)*sddot_c_x  + sigma*sddot_d_x;
430
431     s_r(2)      = (1 - sigma)*s_c_z      + sigma*s_d_z;
432     sdot_r(2)   = (1 - sigma)*sdot_c_z   + sigma*sdot_d_z;
433     sddot_r(2)  = (1 - sigma)*sddot_c_z  + sigma*sddot_d_z;
434
435 else
436     tau2 = tau - (tb + tc);
437     x3 = xA + 0.5*a_x*tb^2 + Vx*tc;
438     z3 = zA + 0.5*a_z*tb^2 + Vz*tc;
439
440     sddot_r(1) = -a_x;
441     sdot_r(1)  = Vx - a_x * tau2;
442     s_r(1)     = x3 + Vx * tau2 - 0.5*a_x * tau2^2;
443
444     sddot_r(2) = -a_z;
445     sdot_r(2)  = Vz - a_z * tau2;
446     s_r(2)     = z3 + Vz * tau2 - 0.5*a_z * tau2^2;
447 end
448 end

```